UNIVERSIDADE FEDERAL DE MINAS GERAIS
ESCOLA DE ENGENHARIA
CURSO DE ENGENHARIA DE SISTEMAS

MONITORAMENTO E OTIMIZAÇÃO DO CONSUMO DE RECURSOS EM APLICATIVOS FLUTTER

Proposta de Ferramenta de Apoio ao Desenvolvimento

Beatriz Vocurca Frade

Orientador: Prof. Ramon Lacerda Marques

Belo Horizonte
2025

Resumo

O monitoramento de desempenho em aplicações móveis depende, na prática, de ferramentas externas que exigem configuração especializada e interpretação manual de dados, uma barreira que restringe sua adoção ao longo do desenvolvimento cotidiano. Este trabalho apresenta o `collector_flutter`, um pacote Dart/Flutter de código aberto que incorpora coleta de métricas, análise heurística e visualização em tempo real diretamente na aplicação monitorada, sem dependências nativas ou ferramentas externas.

O protótipo adota arquitetura de três camadas (*Data*, *Domain* e *Presentation*) baseada nos princípios da *Clean Architecture*, coletando métricas de renderização (FPS, *jank*, percentis P50/P95/P99), uso de memória RSS e *heap*, tráfego HTTP e eventos customizados do desenvolvedor. Um motor heurístico analisa essas métricas a cada ciclo de coleta e gera recomendações de otimização classificadas por severidade, exibidas em um painel visual integrado.

O desenvolvimento seguiu um ciclo de vida iterativo estruturado em quatro fases: revisão bibliográfica, especificação de requisitos, implementação do protótipo e avaliação experimental, as três primeiras concluídas no TCC 1. A validação qualitativa em quatro cenários controlados (*rebuilds* intensivos, carga de rede, alocação de memória e carga combinada) confirmou a operacionalidade do sistema e a pertinência das heurísticas implementadas. A avaliação quantitativa, com análise estatística e comparação com o Android Profiler e o `adb shell dumpsys gfxinfo`, constitui o escopo do TCC 2.

Palavras-chave: Flutter. Monitoramento de desempenho. Profiling. Coleta de métricas em tempo real. Eficiência energética. Sustentabilidade digital. Clean Architecture.

Sumário

Resumo **i**

Lista de Figuras **v**

1 Introdução **1**
1.1 Propósito 1
1.2 Público-Alvo 2
1.3 Motivação e Justificativa 2
1.4 Objetivos 3
1.5 Visão Geral do Documento 3
1.6 Revisão Bibliográfica 4

2 Contexto Social, Ambiental e Ético **7**
2.1 Impactos Socioeconômicos 7
2.2 Impactos Sociais e Inclusão Digital 8
2.3 Impactos Ambientais e Eficiência Computacional 8
2.4 Aspectos Éticos no Monitoramento de Aplicações 9
2.5 Responsabilidade Tecnológica no Desenvolvimento de Software 9
2.6 Síntese 10

3 Metodologia e Desenvolvimento **11**
3.1 Abordagem Metodológica 11
3.1.1 Natureza da Pesquisa 11
3.1.2 Objetivos da Abordagem 11
3.1.3 Justificativa da Escolha Metodológica 12
3.1.4 Ciclo de Vida da Pesquisa e do Desenvolvimento 12
3.2 Cronograma de Execução 15
3.3 Cronograma do TCC 1 15
3.4 Cronograma do TCC 2 15
3.5 Etapas de Desenvolvimento 15
3.5.1 Fase 1 – Revisão Bibliográfica e Fundamentação Teórica 15
3.5.2 Fase 2 – Levantamento de Requisitos e Especificação 19
3.5.3 Fase 3 – Projeto e Implementação do Protótipo 20
3.5.4 Fase 4 – Avaliação Experimental e Validação 21
3.6 Descrição do Protótipo e Arquitetura da Solução 23
3.6.1 Concepção e Premissas de Projeto 23
3.6.2 Arquitetura Geral do Sistema 24

3.6.3 Componentes Principais 25
3.6.4 Interface e Painel de Visualização 26
3.6.5 Regras de Recomendação 28
3.6.6 Demonstração Prática e Disponibilização do Código-Fonte 29
3.7 Critérios de Avaliação 29
3.7.1 Precisão das Métricas Coletadas 30
3.7.2 Overhead Introduzido pelo Sistema 30
3.7.3 Eficácia das Recomendações Geradas 30
3.7.4 Usabilidade e Clareza da Interface 31
3.8 Ferramentas e Ambientes de Desenvolvimento 31
3.8.1 Ambientes de Programação e IDEs 31
3.8.2 SDKs e Dependências Utilizadas 32
3.8.3 Ambientes de Teste e Dispositivos 32
3.8.4 Ferramentas de Referência e Validação 32
3.8.5 Frameworks e Bibliotecas de Apoio 33
3.8.6 Ambientes de Testes Automatizados 33
3.9 Procedimentos de Teste e Coleta de Dados 35
3.9.1 Protocolo de Execução dos Testes 35
3.9.2 Definição dos Cenários Experimentais 35
3.9.3 Coleta de Métricas 36
3.9.4 Métodos de Comparação 36
3.9.5 Análise Estatística dos Resultados 37
3.9.6 Controle Experimental e Reprodutibilidade 37

4 Resultados e Discussão 39
4.1 Desenvolvimento do Protótipo 39
4.1.1 Módulos Implementados 39
4.1.2 Estrutura da Biblioteca 40
4.2 Validação Preliminar da Arquitetura 41
4.2.1 Conformidade Arquitetural 41
4.2.2 Comportamento Observado em Execução 41
4.3 Análise dos Cenários Experimentais Iniciais 42
4.3.1 Cenário 1 — Reconstruções Intensivas de Interface 42
4.3.2 Cenário 2 — Chamadas de Rede Simultâneas 42
4.3.3 Cenário 3 — Alocação Intensiva de Memória 42
4.3.4 Cenário 4 — Carga Combinada 42
4.4 Limitações Identificadas no TCC 1 42
4.5 Resumo do Capítulo 43

5 Conclusões 44
5.1 Considerações Finais 44
5.2 Contribuições do TCC 1 45
5.3 Propostas de Continuidade 45

Lista de Figuras

3.1 Ciclo de vida metodológico adotado (iterativo e incremental). 13
3.2 Arquitetura de três camadas do protótipo `collector_flutter`: Data (coleta), Domain (análise) e Presentation (visualização). 20
3.3 Mapa conceitual da arquitetura geral do protótipo `collector_flutter`. 24
3.4 Painel do aplicativo em condição de desempenho estável, com indicadores verdes e recomendações informativas. 27
3.5 Painel do aplicativo com anomalias detectadas, como FPS baixo e uso de memória elevado. 28
3.6 Ícone do aplicativo de demonstração `Collector Flutter`. 28

Capítulo 1

Introdução

Em fevereiro de 2024, o Flutter ultrapassou a marca de 170 mil repositórios públicos no GitHub [15], consolidando-se como o principal *framework* de desenvolvimento móvel multiplataforma. Esse crescimento acelerado impõe desafios específicos à cadeia de desenvolvimento: equipes migrantes de outros ecossistemas frequentemente se deparam com barreiras técnicas para diagnóstico de desempenho.

Embora ferramentas como *Flutter DevTools*, Android Profiler e Perfetto sejam tecnicamente maduras, sua arquitetura externa ao fluxo de desenvolvimento cria fricções que dificultam a adoção contínua. As principais barreiras identificadas são:

- Requerem saída do ambiente de desenvolvimento (aplicações separadas, WebSocket, conexão USB);
- Demandam configuração manual e interpretação de dados complexos;
- Não fornecem recomendações automáticas de otimização;
- Impõem custo cognitivo elevado em ciclos iterativos rápidos.

Na prática, essas fricções levam equipes a postergar o diagnóstico de performance para fases tardias do projeto, quando regressões já podem ter impactado o usuário final.

Este trabalho propõe o `collector_flutter`: um pacote Dart de código aberto que integra coleta de métricas, análise heurística e visualização diretamente na aplicação monitorada. O diferencial reside na **acessibilidade**: basta adicionar o pacote e realizar três chamadas de código para ativar o painel de diagnóstico, que opera como uma tela comum do aplicativo, sem configurações externas.

1.1 Propósito

O propósito deste trabalho é projetar, implementar e avaliar preliminarmente o `collector_flutter`, uma biblioteca Dart para monitoramento de desempenho em tempo real em aplicações Flutter. O pacote coleta métricas de renderização (FPS, *jank*, percentis de tempo de quadro), uso de memória RSS e *heap*, tráfego HTTP e eventos customizados definidos pelo desenvolvedor. Um motor heurístico analisa essas métricas a cada ciclo de coleta e exibe recomendações de otimização classificadas por severidade em um painel visual integrado.

A proposta não substitui ferramentas de diagnóstico especializadas como o *Flutter DevTools* ou o Android Profiler. Seu papel é diferente: funcionar como um termômetro contínuo durante o desenvolvimento, capaz de sinalizar regressões de performance no mesmo momento em que o código é exercitado — sem exigir que o desenvolvedor alterne de contexto para uma ferramenta externa.

1.2 Público-Alvo

O trabalho se destina a três grupos principais:

- Desenvolvedores Flutter que precisam de feedback de performance integrado ao fluxo de desenvolvimento, especialmente em equipes pequenas sem recursos para ciclos de *profiling* dedicados;
- Pesquisadores de engenharia de software interessados no problema de instrumentação não-intrusiva em frameworks reativos — em particular na relação entre frequência de coleta, precisão das métricas e *overhead* introduzido;
- Estudantes de computação e sistemas que buscam um exemplo concreto de como aplicar princípios de *Clean Architecture* e programação reativa em Dart para construir uma biblioteca com separação clara entre coleta, análise e apresentação de dados.

1.3 Motivação e Justificativa

A relação entre desempenho e consumo energético em aplicações móveis é documentada pela literatura desde o trabalho seminal de Pathak *et al.* [1], que demonstrou que componentes de interface e operações de rede respondem pela maior parte do consumo energético em dispositivos Android. Zhang *et al.* [19] aplicaram metodologia semelhante ao ecossistema Flutter e identificaram que excessivos de widgets — uma característica inerente ao modelo reativo do framework — podem elevar o consumo de CPU em até 35% em relação a implementações equivalentes com React Native otimizado.

Esses dados têm implicações práticas diretas: um aplicativo com renderização ineficiente drena a bateria mais rápido, eleva a temperatura do dispositivo e, em aparelhos de entrada — que ainda representam a maioria do mercado em economias emergentes [22] — pode tornar o serviço inutilizável. A sustentabilidade de um ecossistema digital passa, portanto, pela capacidade dos desenvolvedores de identificar e corrigir ineficiências antes que estas cheguem ao usuário final.

O problema é que a detecção precoce de regressões de performance requer instrumentação contínua, e as ferramentas disponíveis impõem uma barreira de configuração que raramente é atravessada durante o desenvolvimento rotineiro [20]. O *Flutter DevTools* fornece os dados necessários, mas exige que o desenvolvedor abra uma aplicação separada, conecte ao processo em execução e interprete gráficos sem qualquer indicação automática do que representa um comportamento anômalo. Essa lacuna — entre a disponibilidade dos dados e a capacidade de agir sobre eles no momento certo — é o problema central que este trabalho endereça.

1.4 Objetivos

O objetivo geral deste trabalho é desenvolver e avaliar experimentalmente o `collector_flutter`, demonstrando que é possível construir um sistema de monitoramento de desempenho embutido para Flutter com *overhead* inferior a 5% sobre o tempo médio de renderização de quadros e precisão comparável, dentro de margem de 10%, às ferramentas de referência do ecossistema.

Os objetivos específicos são:

1. Realizar revisão bibliográfica sistemática sobre monitoramento de recursos, eficiência energética e ferramentas de *profiling* em aplicações móveis, com foco no ecossistema Flutter.

2. Identificar e documentar as limitações das ferramentas de diagnóstico existentes quanto à integração ao fluxo de desenvolvimento e à geração de recomendações automáticas.

3. Projetar uma arquitetura modular de três camadas que separe coleta, análise e apresentação de métricas, garantindo baixo acoplamento e extensibilidade.

4. Implementar os módulos de coleta (FPS, memória, rede, eventos), análise heurística e visualização em tempo real como um pacote Dart publicável.

5. Validar qualitativamente o protótipo em quatro cenários experimentais controlados no TCC 1 e quantitativamente, com análise estatística, no TCC 2.

6. Mensurar o *overhead* introduzido pelo monitoramento e a precisão das métricas coletadas em comparação com Android Profiler e `adb shell dumpsys gfxinfo`.

7. Disponibilizar o protótipo como código aberto, com documentação técnica suficiente para reprodução e extensão por outros pesquisadores.

1.5 Visão Geral do Documento

O Capítulo 2 apresenta a fundamentação teórica, reunindo a revisão bibliográfica sobre monitoramento de recursos e ferramentas de diagnóstico existentes, e o contexto de humanidades que situa o trabalho nas dimensões de sustentabilidade digital, impacto social e responsabilidade ética.

O Capítulo 3 descreve a metodologia adotada, incluindo a natureza da pesquisa, o ciclo de vida iterativo, a arquitetura do protótipo, os requisitos funcionais e não funcionais, a especificação dos módulos implementados e os procedimentos experimentais planejados.

O Capítulo 4 apresenta os resultados do TCC 1: o estado de implementação de cada módulo, a validação arquitetural preliminar e as observações qualitativas dos quatro cenários experimentais.

O Capítulo 5 consolida as conclusões do TCC 1 e delineia o escopo do TCC 2, incluindo a execução formal dos experimentos, análise estatística e comparação com ferramentas de referência.

1.6 Revisão Bibliográfica

A construção de uma ferramenta de monitoramento de desempenho para Flutter requer embasamento em três eixos complementares: (i) a natureza das métricas de execução em plataformas móveis e as técnicas para capturá-las; (ii) as soluções existentes e suas limitações; e (iii) as abordagens acadêmicas que investigam o problema de instrumentação não-intrusiva e eficiência energética. Esta seção sintetiza criticamente esses eixos e identifica a lacuna que este trabalho busca preencher.

Métricas de Desempenho e Técnicas de Coleta em Aplicações Móveis

O estudo de métricas de desempenho em dispositivos móveis consolidou-se com o trabalho de Pathak *et al.* [1], que demonstrou, por meio de modelagem energética em nível de componente, que chamadas de rede e operações de renderização de interface respondem pela maior parcela do consumo energético em smartphones Android — resultado que direcionou uma geração de pesquisas sobre instrumentação eficiente de aplicações móveis.

No contexto de *profiling* de desempenho, Liu *et al.* [2] investigaram técnicas de rastreamento de chamadas de sistema e eventos de renderização de quadros, mostrando que a correlação entre métricas de baixo nível (latência de sistema, tempo de GPU) e métricas perceptíveis ao usuário (FPS, responsividade) é viável e relevante para diagnóstico automatizado. Essa correlação é a base do motor heurístico proposto no presente trabalho.

Pesquisas mais recentes confirmam que ineficiências de renderização têm impacto energético mensurável. Zhang *et al.* [19] quantificaram que aplicações Flutter com *rebuilds* frequentes consomem até 35% mais CPU do que implementações equivalentes com React Native otimizado, atribuindo a diferença ao custo de reconciliação da árvore de widgets. He *et al.* [3] ampliaram essa análise comparativa para incluir tempo de renderização, uso de memória e *overhead* de ponte nativa, documentando diferenças estatisticamente significativas entre os frameworks.

Estudos com foco em automação da detecção de anomalias seguem uma linha complementar. Banerjee *et al.* [4] propuseram um sistema de monitoramento em tempo de execução capaz de identificar padrões de comportamento anômalo sem modificações no código da aplicação — uma propriedade que o `collector_flutter` busca preservar por meio da coleta via `SchedulerBinding` e VM Service, sem necessidade de anotações explícitas nos widgets monitorados.

Do ponto de vista energético, os trabalhos de Hao *et al.* [5] e Li e Halfond [6] estabeleceram que o consumo de energia em aplicações Android pode ser estimado a partir de métricas de execução observáveis, sem instrumentação de hardware dedicada. Essa abordagem é relevante para os planos de implementação de métricas de energia no TCC 2.

Ferramentas de Monitoramento Existentes

O ecossistema de desenvolvimento móvel dispõe de ferramentas maduras de *profiling*, mas com características que limitam sua adoção contínua durante o desenvolvimento. A Tabela 1.1 apresenta um comparativo estruturado entre as principais soluções disponíveis.

A análise da tabela revela um padrão consistente: ferramentas externas como o DevTools e o Android Profiler fornecem dados de alta qualidade, mas exigem saída do ambiente de

Tabela 1.1: Comparativo de ferramentas de monitoramento e análise de desempenho.

Ferramenta	**Principais Funcionalidades**	**Limitações Relevantes**
Ferramentas e Soluções de Monitoramento		
Flutter DevTools	Inspeção de widgets, análise de memória, *timeline* de frames e perfil de CPU.	Opera como aplicação separada; requer conexão via WebSocket ao processo; ausência de recomendações automáticas; dependência do modo *debug/profile*.
Perfetto (AOSP)	Rastreamento detalhado de processos e eventos do sistema Android com suporte a múltiplas camadas.	Configuração manual de sessões de rastreamento; interpretação de traços binários; alto custo cognitivo; não integrado ao fluxo de desenvolvimento.
Android Profiler	Monitoramento em tempo real de CPU, memória, rede e energia diretamente no Android Studio.	Restrito a Android; requer conexão USB em modo *profile*; integração limitada com frameworks multiplataforma.
Xcode Instruments	Coleta avançada de métricas de energia, CPU e renderização para dispositivos iOS.	Exclusivo ao ecossistema Apple; coleta manual de dados; elevado *overhead* em algumas configurações.
Firebase Performance	Monitoramento automático de tempos de carregamento e chamadas de rede em produção.	Granularidade limitada; sem métricas de renderização ou CPU; voltado a produção, não ao desenvolvimento.
Flipper (Meta)	Ferramenta de depuração para React Native com análise de rede e layout.	Compatibilidade limitada com Flutter; requer integrações adicionais não mantidas pela comunidade.
LeakCanary	Detecção automatizada de vazamentos de memória em Android nativo.	Foco exclusivo em memória; sem métricas de renderização, CPU ou rede.

desenvolvimento, configuração explícita e interpretação manual dos resultados. Cheng e Wang [18] documentaram que esse custo de configuração é um dos principais fatores que levam desenvolvedores a realizar diagnósticos de performance apenas em fases tardias do projeto, quando regressões já chegaram à produção.

Johnson [20] identificou que a ausência de recomendações automáticas é a limitação mais citada pelos desenvolvedores ao descrever sua experiência com ferramentas de *profiling* — o que aponta para uma oportunidade de contribuição: sistemas que não apenas coletam dados, mas os interpretam e comunicam de forma acionável.

Estudos Acadêmicos Relacionados

A pesquisa em monitoramento de desempenho para aplicações Flutter é recente. Santos [8] realizou avaliação comparativa de três abordagens de estruturação da árvore de widgets e documentou reduções de até 20% no número de *rebuilds* ao adotar `const` construtores e `ValueNotifier` em substituição a `setState` global. Silva [7] conduziu experimento semelhante em aplicações Android nativas e encontrou correlação estatisticamente significativa entre frequência de *rebuilds* e consumo de CPU.

Rodrigues [9] investigou frameworks de monitoramento para aplicações móveis e identificou que soluções embutidas — que coletam métricas sem interromper o processo de desenvolvimento — apresentam maior taxa de adoção do que ferramentas externas, independentemente da riqueza dos dados fornecidos. Esse resultado orienta diretamente a decisão de projeto do `collector_flutter`: o valor da acessibilidade supera o valor da granularidade para o caso de uso de diagnóstico contínuo durante o desenvolvimento.

Em perspectiva mais ampla, o levantamento sistemático de Kumar e Gupta [21] sobre técnicas de otimização de recursos em aplicações móveis categorizou estratégias em quatro dimensões — renderização, memória, rede e energia — e concluiu que sistemas de recomendação automatizada representam o elo mais fraco da cadeia: a maioria das ferramentas existentes diagnostica, mas não prescreve. O motor heurístico do `collector_flutter` é uma resposta direta a essa lacuna.

Lacuna de Pesquisa

A revisão evidencia que as ferramentas de *profiling* disponíveis para Flutter são tecnicamente maduras, mas que sua arquitetura externa ao processo de execução cria uma fricção de uso que limita sua adoção contínua. As abordagens acadêmicas investigam métricas, técnicas de coleta e impacto energético, mas raramente propõem sistemas completos — da coleta à recomendação — que operem de forma embutida e com *overhead* explicitamente controlado.

O presente trabalho propõe o `collector_flutter` como contribuição a essa lacuna: uma biblioteca que integra coleta automática de métricas, análise heurística e geração de recomendações em um único componente, sem dependências nativas ou configuração externa, com *overhead* alvo inferior a 5% sobre o tempo médio de renderização de quadros. A hipótese central, a ser verificada no TCC 2, é que esse conjunto de características produz uma ferramenta com taxa de adoção superior às alternativas externas existentes, mensurável por meio de avaliação de usabilidade com desenvolvedores Flutter.

Capítulo 2

Contexto Social, Ambiental e Ético

2.1 Impactos Socioeconômicos

Eficiência computacional tem valor econômico mensurável. Em infraestrutura de servidores, a relação é direta: ciclos de CPU poupados são custos de nuvem reduzidos. Em dispositivos móveis, o mecanismo é diferente mas igualmente concreto: aplicações mais leves consomem menos bateria, reduzem a frequência de carregamentos e prolongam a vida útil dos dispositivos [18]. Em mercados como Brasil e Índia, onde o custo de um aparelho equivale a dois a três meses de salário mínimo, a longevidade do hardware é economicamente relevante para o usuário final.

Do ponto de vista social, aplicações mal otimizadas reforçam desigualdades de acesso. Um serviço de saúde ou de educação que funciona bem em um Pixel 8 mas gera *jank* constante em um dispositivo de entrada cria uma experiência de segunda classe para usuários que já possuem menor capacidade de escolha. Otimização de performance, nesse contexto, é também uma questão de equidade no acesso a serviços digitais.

Ferramentas de software raramente são neutras em seus efeitos. Uma biblioteca de monitoramento de desempenho para Flutter — aparentemente um artefato técnico circunscrito ao ecossistema de desenvolvimento — carrega consigo decisões de projeto com implicações que transcendem a eficiência computacional: para quem a ferramenta é acessível, quais dados ela coleta e como, e em que medida ela contribui para práticas de desenvolvimento mais conscientes do impacto ambiental e social das aplicações produzidas. Este capítulo situa o `collector_flutter` nessas dimensões mais amplas.

Sustentabilidade Digital e o Custo Energético do Software

A relação entre eficiência de software e consumo energético ganhou escala com a proliferação de dispositivos móveis. O relatório de sustentabilidade do Google [22] estima que melhorias na eficiência de renderização, gerenciamento de cache e políticas de rede podem reduzir o consumo energético de uma aplicação móvel típica em até 40% sem degradação perceptível da experiência do usuário. O IEEE [23] situa o setor de TIC — incluindo dispositivos móveis — em aproximadamente 1,8% das emissões globais de carbono, com projeção de crescimento sustentado ao longo da década.

Esses números têm uma implicação prática específica para o desenvolvimento de software: ineficiências locais se multiplicam pela escala. Um *rebuild* desnecessário de widget

que consome 2 ms de CPU extras, distribuído por 50 milhões de execuções diárias em uma base de usuários moderada, representa centenas de gigawatts-hora de energia desperdiçada por ano. Cheng e Wang [18] demonstraram que reduções de chamadas de rede redundantes e otimização de operações gráficas produzem economias energéticas mensuráveis mesmo em escala de dispositivo individual — efeito que, somado, é ambientalmente relevante.

O `collector_flutter` insere-se nessa perspectiva ao tornar visível, para o desenvolvedor, o custo computacional das escolhas de implementação. A identificação de *jank* recorrente, de tendências de crescimento de memória ou de volume excessivo de requisições de rede não serve apenas para melhorar a experiência do usuário: serve também para reduzir o consumo energético da aplicação resultante, contribuindo para a dimensão de sustentabilidade digital da engenharia de software.

2.2 Impactos Sociais e Inclusão Digital

Desempenho não é uma preocupação uniforme ao longo da pirâmide socioeconômica. Aplicações que funcionam bem em dispositivos de alta capacidade — Pixels, iPhones, Galaxy S — frequentemente degradam de forma severa em aparelhos de entrada que ainda representam a maioria absoluta do mercado em economias em desenvolvimento [22]. Essa assimetria tem consequências concretas: serviços de saúde, educação e comunicação que se tornam praticamente inutilizáveis em dispositivos de menor custo criam uma experiência de segunda classe para usuários que já possuem menor poder de escolha.

A otimização de performance em aplicações Flutter é, portanto, também uma questão de equidade de acesso. Uma aplicação que consome 40% menos CPU e memória não é apenas mais eficiente — é mais inclusiva, porque opera adequadamente em uma gama mais ampla de dispositivos. Ferramentas que facilitam a identificação e correção de ineficiências durante o desenvolvimento contribuem indiretamente para essa inclusão, ao reduzir a barreira técnica entre uma aplicação funcionalmente correta e uma aplicação acessível para todos os usuários potenciais.

O `collector_flutter` foi projetado para operar sem restrições de plataforma ou dispositivo. A coleta de métricas via `SchedulerBinding` e VM Service funciona igualmente em aparelhos de alta e baixa capacidade; as heurísticas de recomendação são calibradas para detectar comportamentos que afetam desempenho em dispositivos com restrições de hardware. Isso não torna a ferramenta um instrumento de inclusão digital por si só, mas alinha suas escolhas de projeto com esse valor.

2.3 Impactos Ambientais e Eficiência Computacional

O ciclo de vida de um dispositivo móvel tem dois determinantes principais: a durabilidade física do hardware e a compatibilidade do software disponível com suas especificações. Aplicações cada vez mais exigentes acceleram a obsolescência percebida dos dispositivos: quando um smartphone de três anos não consegue executar as versões mais recentes dos aplicativos principais sem *jank* severo, o incentivo para descartá-lo aumenta — mesmo que o hardware esteja fisicamente íntegro.

Resíduos eletrônicos constituem uma das correntes de lixo de crescimento mais rápido no mundo [24]. Uma contribuição modesta, mas concreta, da engenharia de software para

mitigar esse problema é o desenvolvimento de aplicações que continuem funcionando adequadamente em dispositivos mais antigos. Ferramentas de monitoramento que tornam visível o custo computacional do software — e que geram recomendações práticas para reduzi-lo — apoiam esse objetivo.

2.4 Aspectos Éticos no Monitoramento de Aplicações

Sistemas de monitoramento de desempenho coletam informações sobre o comportamento de aplicações em execução. Em contextos de produção, onde o monitoramento ocorre sobre dispositivos de usuários reais, essa coleta levanta questões éticas legítimas: quais dados são coletados, por quanto tempo, por quem podem ser acessados e para quais finalidades podem ser utilizados.

O `collector_flutter` foi projetado com o princípio de minimização de dados: ele coleta exclusivamente métricas de desempenho do sistema — tempo de quadro, uso de memória, contagem de requisições, eventos definidos pelo desenvolvedor. Nenhum dado de identificação de usuário, conteúdo de comunicação ou informação pessoal é registrado. As métricas existem apenas em memória durante a sessão de uso da ferramenta e, na versão atual, não são transmitidas a servidores externos.

Essa delimitação não é apenas uma escolha ética — é também uma escolha técnica deliberada. Um sistema de monitoramento que coleta mais dados do que o necessário para seus objetivos de diagnóstico introduz riscos de privacidade desnecessários e aumenta o *overhead* sem benefício correspondente para o desenvolvedor. O princípio de minimização de dados serve, portanto, tanto à ética quanto à eficiência.

A disponibilização do código-fonte como software aberto contribui adicionalmente para a transparência: qualquer desenvolvedor pode auditar o que é e o que não é coletado, verificar as heurísticas de recomendação e propor melhorias. Essa abertura é coerente com os padrões de reprodutibilidade científica recomendados por Wohlin *et al.* [28] para pesquisas experimentais em engenharia de software.

2.5 Responsabilidade Tecnológica no Desenvolvimento de Software

A noção de responsabilidade tecnológica — a ideia de que desenvolvedores e pesquisadores são corresponsáveis pelos impactos das soluções que produzem — é cada vez mais central na engenharia de software contemporânea. Isso se traduz, na prática, em considerar não apenas a correção funcional e o desempenho técnico de um sistema, mas também suas implicações de acessibilidade, consumo de recursos, privacidade e longevidade.

Este trabalho é uma contribuição técnica — um pacote Dart com arquitetura bem definida e protocolo de avaliação experimental rigoroso. Mas é também uma tomada de posição: de que diagnóstico de performance deveria ser acessível ao desenvolvedor que trabalha em um projeto de equipe pequena, sem orçamento para ciclos de *profiling* dedicados; de que eficiência computacional é um critério de qualidade que deveria ser monitorado continuamente, não apenas em fases de otimização tardia; e de que ferramentas que democratizam o acesso a esse diagnóstico contribuem, de forma modesta mas real, para um ecossistema digital mais

eficiente e mais equitativo.

2.6 Síntese

O monitoramento de desempenho em aplicações Flutter, enquanto problema técnico, é bem delimitado: coletar métricas, analisá-las e comunicar resultados ao desenvolvedor. Enquanto problema de engenharia com dimensões sociais e ambientais, é mais amplo: tornar esse diagnóstico acessível reduz o consumo energético das aplicações produzidas, amplia sua compatibilidade com dispositivos de menor capacidade e contribui para práticas de desenvolvimento mais conscientes de seu impacto. O `collector_flutter` é uma resposta ao primeiro problema que busca contribuir, de forma instrumental, para o segundo.

Capítulo 3

Metodologia e Desenvolvimento

3.1 Abordagem Metodológica

3.1.1 Natureza da Pesquisa

A presente pesquisa caracteriza-se como um estudo de natureza **aplicada**, pois visa desenvolver e validar um *protótipo funcional* de monitoramento de desempenho para aplicações Flutter. Segundo Gil [25], pesquisas aplicadas têm como propósito "gerar conhecimentos voltados à aplicação prática, dirigidos à solução de problemas específicos".

Nesse contexto, o protótipo `collector_flutter` busca oferecer uma solução técnica que permita a coleta, análise e interpretação de métricas de execução em tempo real, contribuindo para o aprimoramento da eficiência e da sustentabilidade digital de aplicativos móveis.

A pesquisa também apresenta abordagem **experimental**, uma vez que envolve a manipulação controlada de variáveis e a observação empírica dos resultados em ambientes reais. Conforme Lakatos e Marconi [26], o método experimental é adequado quando se pretende "testar hipóteses mediante o controle e a observação dos efeitos provocados por determinadas condições".

3.1.2 Objetivos da Abordagem

A adoção de uma abordagem aplicada e experimental permite atender a objetivos técnicos e científicos específicos, entre os quais se destacam:

- **Demonstrar a viabilidade técnica** de um sistema de monitoramento autônomo e totalmente embutido no ecossistema Flutter, sem dependência de ferramentas nativas externas (Android Developers [10]);
- **Avaliar o impacto da coleta de métricas** sobre o desempenho da aplicação, mensurando o *overhead* introduzido pelo processo de monitoramento;
- **Mensurar a precisão dos dados coletados**, comparando-os às medições obtidas por ferramentas oficiais como Android Profiler e Xcode Instruments (AOSP [11]);
- **Validar a eficácia das recomendações heurísticas**, verificando se sua aplicação gera melhorias tangíveis no desempenho e no consumo de recursos;

- **Promover a reprodutibilidade científica**, disponibilizando publicamente o código-fonte e a documentação técnica do protótipo, conforme recomendam Wohlin *et al.* [28].

Esses objetivos orientam tanto o delineamento experimental quanto as etapas de desenvolvimento do sistema, assegurando coerência entre a fundamentação teórica, o projeto técnico e a avaliação empírica dos resultados.

3.1.3 Justificativa da Escolha Metodológica

A escolha de uma metodologia de natureza aplicada e abordagem experimental fundamenta-se na necessidade de validar empiricamente o comportamento de sistemas de monitoramento em contextos reais de execução. Conforme destaca Yin [27], abordagens experimentais e estudos de caso são apropriados quando o pesquisador busca compreender relações de causa e efeito entre variáveis observáveis em ambientes complexos como é o caso de aplicações móveis em execução sobre diferentes plataformas e dispositivos.

No caso do `collector_flutter`, a observação direta de variáveis como *frame time*, reconstruções de widgets e uso de memória é indispensável para a avaliação de sua precisão e impacto. Seguindo a orientação de Meyer e Krause [16], a análise experimental é essencial para estudos de engenharia de software focados em desempenho, pois permite associar métricas quantitativas a efeitos percebidos pelo usuário final.

Assim, a abordagem metodológica proposta combina:

- fundamentos teóricos sobre desempenho, eficiência e sustentabilidade digital;
- experimentação controlada em dispositivos Android e iOS;
- análise comparativa dos resultados obtidos com ferramentas nativas de referência.

Essa integração entre teoria e prática garante a robustez científica do estudo e o caráter inovador do sistema proposto, permitindo que ele seja avaliado, reproduzido e aprimorado em diferentes contextos de desenvolvimento.

3.1.4 Ciclo de Vida da Pesquisa e do Desenvolvimento

O processo adotado neste trabalho segue um **ciclo de vida iterativo e incremental**, característico de projetos de engenharia de software com validação empírica, combinando desenvolvimento de protótipo e experimentação controlada. Esse ciclo foi estruturado com base em princípios de engenharia de software (Pressman e Maxim [29]; Sommerville [31]) e em diretrizes de experimentação em engenharia de software (Wohlin *et al.* [28]), garantindo rastreabilidade entre objetivos, decisões de projeto e resultados.

A Figura 3.1 representa o ciclo metodológico adotado, organizado em fases com **entradas**, **atividades**, **artefatos gerados** e **critérios de término**. O ciclo contempla dois momentos do trabalho: (i) **TCC 1**, voltado à fundamentação e ao protótipo inicial; e (ii) **TCC 2**, voltado à consolidação, testes experimentais e análise estatística.

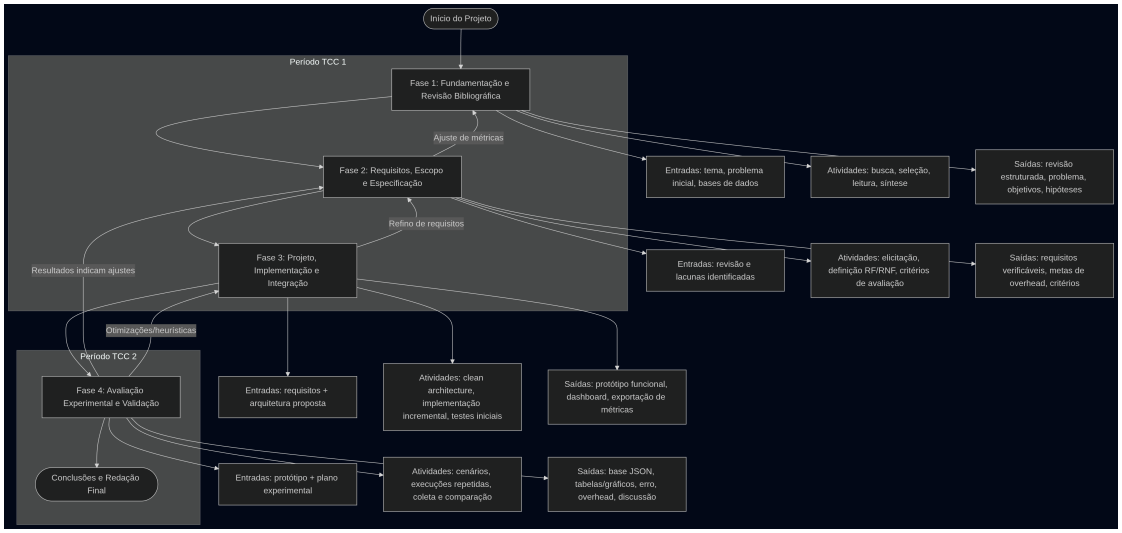

Figura 3.1: Ciclo de vida metodológico adotado (iterativo e incremental).

Visão Geral das Fases e Entregáveis

O ciclo de vida foi operacionalizado em quatro fases principais, descritas a seguir. Para cada fase, definem-se claramente: (i) o objetivo, (ii) as atividades, (iii) os artefatos produzidos e (iv) o critério de conclusão.

Fase 1 — Fundamentação e Revisão Bibliográfica **Objetivo:** construir o embasamento teórico sobre desempenho, profiling, métricas e eficiência energética em aplicações móveis, delimitando o problema e justificando a solução proposta. **Atividades:** busca sistemática por palavras-chave, leitura e seleção de estudos, organização temática e consolidação de lacunas na literatura. **Artefatos/saídas:** (i) revisão bibliográfica estruturada, (ii) definição do problema, (iii) objetivos e hipóteses preliminares. **Critério de conclusão:** existência de um referencial teórico suficiente para sustentar as métricas monitoradas e as escolhas de projeto do protótipo.

Fase 2 — Requisitos, Escopo e Especificação **Objetivo:** transformar o problema de pesquisa em requisitos verificáveis e critérios de sucesso, descrevendo claramente o que o protótipo deve fazer e quais limites deve respeitar. **Atividades:** elicitação e análise de requisitos, definição de critérios de qualidade e operacionalização das métricas (o que medir e como medir). **Artefatos/saídas:** (i) requisitos funcionais e não funcionais, (ii) critérios de avaliação, (iii) definição de metas (ex.: limite de *overhead*). **Critério de conclusão:** requisitos claros e mensuráveis, capazes de orientar implementação e avaliação experimental.

Fase 3 — Projeto, Implementação e Integração do Protótipo **Objetivo:** projetar e implementar o `collector_flutter` de forma modular e testável, garantindo integração com aplicações Flutter e capacidade de coleta/visualização local. **Atividades:** definição da arquitetura (camadas e módulos), implementação incremental das funcionalidades, integração entre módulos e testes iniciais. **Artefatos/saídas:** (i) protótipo funcional, (ii) arquitetura documentada (diagramas e componentes), (iii) painel (*dashboard*) integrado, (iv) registro e

exportação de métricas. **Critério de conclusão:** protótipo executável, capaz de coletar e exibir métricas em tempo real em ambiente controlado.

Fase 4 — Avaliação Experimental, Análise e Validação **Objetivo:** validar empiricamente precisão, impacto e utilidade do protótipo, comparando resultados com ferramentas de referência e aplicando análise estatística. **Atividades:** definição dos cenários experimentais, execução repetida dos testes, coleta automatizada, análise descritiva e inferencial (quando aplicável), comparação com ferramentas oficiais. **Artefatos/saídas:** (i) base de dados experimental (JSON), (ii) tabelas e gráficos, (iii) métricas de erro/precisão e *overhead*, (iv) discussão de resultados e limitações. **Critério de conclusão:** evidências empíricas suficientes para aceitar/refutar hipóteses e responder aos objetivos do trabalho.

Iterações e Rastreabilidade

Embora as fases sejam apresentadas de forma sequencial, o ciclo adotado é **iterativo**: resultados parciais de testes e integração podem motivar ajustes em requisitos, arquitetura ou heurísticas, desde que tais mudanças sejam registradas e justificadas. Para garantir rastreabilidade, cada funcionalidade do protótipo foi associada a: (i) um requisito (RF/RNF), (ii) um módulo da arquitetura e (iii) um critério de avaliação experimental, permitindo ligação direta entre o que foi proposto, implementado e validado.

Relação com o Cronograma do TCC 1 e TCC 2

O ciclo metodológico foi distribuído ao longo do cronograma em dois macroperíodos:

- **TCC 1:** foco nas Fases 1, 2 e início da Fase 3, resultando em fundamentação teórica consolidada e protótipo inicial funcional.
- **TCC 2:** foco na conclusão da Fase 3 e execução completa da Fase 4, resultando em experimentação, análise estatística e redação final da monografia.

Essa estrutura torna explícito o ciclo de vida adotado e define o que se espera em cada etapa do processo, atendendo ao requisito de maior rigor e sistematização metodológica.

3.2 Cronograma de Execução

Este capítulo apresenta o cronograma das atividades planejadas para o desenvolvimento do Trabalho de Conclusão de Curso, dividido em duas etapas complementares: **TCC 1**, voltado à fundamentação teórica, definição do problema e desenvolvimento inicial do protótipo; e **TCC 2**, dedicado à implementação final, testes experimentais, análise de resultados e redação conclusiva da monografia. O conjunto das atividades planejadas para o TCC 1 e TCC 2 segue uma sequência lógica de **pesquisa, desenvolvimento e validação**. O primeiro ciclo corresponde à fundamentação teórica e concepção do protótipo, enquanto o segundo concentra-se na experimentação e consolidação dos resultados obtidos.
Esse planejamento garante uma condução estruturada, progressiva e reprodutível do projeto, assegurando que cada etapa contribua de forma integrada para o alcance dos objetivos científicos e tecnológicos estabelecidos.

3.3 Cronograma do TCC 1

O TCC 1 contempla o período de **agosto a novembro de 2025** e tem como objetivo consolidar a fundamentação teórica, definir a arquitetura do sistema e apresentar um protótipo inicial funcional. O Quadro 3.1 apresenta a distribuição das atividades planejadas.

3.4 Cronograma do TCC 2

O TCC 2 abrangerá o período de **março a julho de 2026**, com foco na expansão do protótipo, execução dos testes experimentais, análise comparativa dos resultados e finalização da monografia. Essa etapa representa a consolidação prática e científica do estudo iniciado no TCC 1.

3.5 Etapas de Desenvolvimento

O desenvolvimento do protótipo `collector_flutter` foi conduzido de forma iterativa e incremental, seguindo princípios de engenharia de software experimental conforme orientam Wohlin *et al.* [28] e Pressman e Maxim [29]. As etapas foram planejadas de modo a integrar o rigor técnico com a experimentação empírica, garantindo a rastreabilidade entre objetivos, decisões de projeto e resultados obtidos.

3.5.1 Fase 1 – Revisão Bibliográfica e Fundamentação Teórica

Na primeira fase, foi realizada uma revisão bibliográfica sistemática sobre métricas de desempenho, consumo energético e práticas de otimização em aplicações móveis. Segundo Kitchenham e Charters [30], revisões sistemáticas são fundamentais para identificar lacunas na literatura e fundamentar decisões de projeto com base em evidências.

Foram consultadas bases de dados como IEEE Xplore, ACM Digital Library e SpringerLink, utilizando termos relacionados a *software profiling*, *mobile performance*, *Flutter*

Tabela 3.1: Cronograma do TCC 1 (Agosto a Novembro de 2025).

Período	**Atividade**	**Descrição**
TCC 1 - Agosto a Novembro/2025		
Semanas 1–3 (04–22 ago)	Revisão Bibliográfica e Definição do Problema	Revisão sistemática sobre monitoramento de desempenho, consumo de recursos e eficiência energética em aplicativos móveis. Definição detalhada do problema de pesquisa e dos objetivos específicos.
Semana 4 (25–29 ago)	Consolidação da Proposta	Discussão com o orientador e entrega da versão final da *Proposta de TCC*.
Semanas 5–6 (01–12 set)	Levantamento de Ferramentas e Tecnologias	Avaliação de bibliotecas e APIs para coleta de métricas em Flutter. Análise comparativa entre soluções existentes como *DevTools*, *Perfetto* e *Flipper*.
Semanas 7–9 (15 set–03 out)	Modelagem e Arquitetura da Solução	Definição da arquitetura, fluxos de dados, diagramas de componentes e métricas a serem monitoradas.
Semana 10 (06 out)	Entrega do Documento de Visão	Entrega do *Documento de Visão Geral do Projeto*.
Semanas 11–13 (07–31 out)	Implementação do Protótipo Inicial	Desenvolvimento dos módulos de coleta de métricas, interface básica e integração com o aplicativo de teste. Testes preliminares de funcionalidade.
Semanas 14–16 (03–21 nov)	Redação da Monografia e Testes Finais	Elaboração inicial da monografia, análise dos resultados preliminares, documentação da metodologia e ajustes no protótipo.
Semana 17 (24–29 nov)	Revisão e Seminário Final	Revisão geral, formatação e entrega do **Texto Inicial da Monografia**, seguida da preparação e apresentação do seminário final do TCC 1.

Tabela 3.2: Cronograma do TCC 2 (Março a Julho de 2026).

Período	**Atividade**	**Descrição**
TCC 2 - Março a Julho/2026		
Semanas 1–3 (03–21 mar)	Revisão e Planejamento Inicial	Revisão do trabalho anterior, consolidação dos objetivos e definição do plano de testes e métricas de avaliação do protótipo.
Semanas 4–6 (24 mar–11 abr)	Aperfeiçoamento do Protótipo	Atualização do `collector_flutter` com novos módulos de análise, otimização das rotinas de coleta e implementação do painel de visualização (*dashboard*) com regras heurísticas aprimoradas.
Semanas 7–9 (14 abr–02 mai)	Execução dos Testes Experimentais	Realização de testes em dispositivos reais, medindo consumo de CPU, memória, rede e energia. Comparação com ferramentas de referência como Android Profiler, *Perfetto* e *Instruments*.
Semanas 10–12 (05–23 mai)	Análise dos Resultados	Análise estatística e interpretação dos dados coletados. Avaliação de precisão, desempenho e impacto das recomendações sugeridas pelo protótipo.
Semanas 13–15 (26 mai–13 jun)	Consolidação e Documentação	Elaboração dos gráficos comparativos, documentação detalhada dos experimentos e revisão textual conforme as normas da ABNT.
Semanas 16–18 (16 jun–04 jul)	Redação Final e Defesa	Redação final da monografia, revisão orientada, ajustes de formatação e preparação da apresentação e defesa pública do TCC 2.

metrics, e *energy efficiency*. Essa etapa permitiu consolidar o arcabouço teórico necessário à definição dos indicadores de desempenho a serem monitorados e das heurísticas de recomendação incorporadas ao protótipo.

3.5.2 Fase 2 – Levantamento de Requisitos e Especificação

A segunda fase envolveu o levantamento e a análise dos requisitos funcionais e não funcionais do sistema, de forma a alinhar a proposta às necessidades práticas de desenvolvedores Flutter. Seguindo o modelo sugerido por Sommerville [31], o processo de elicitação de requisitos considerou aspectos de usabilidade, integração, portabilidade e impacto sobre o desempenho.

Requisitos Funcionais

Os requisitos funcionais definem as operações que o sistema deve realizar. Entre os principais, destacam-se:

- RF1 – Coletar métricas de desempenho em tempo real, incluindo FPS, reconstruções de widgets, uso de memória e duração de quadros;
- RF2 – Processar e correlacionar as métricas coletadas, identificando padrões de degradação de desempenho;
- RF3 – Gerar recomendações heurísticas de otimização baseadas em boas práticas do ecossistema Flutter;
- RF4 – Exibir métricas e recomendações em um painel visual integrado ao aplicativo;
- RF5 – Permitir a simulação de cenários de carga e coleta sob condições controladas.

Requisitos Não Funcionais

Os requisitos não funcionais descrevem as propriedades de qualidade esperadas do sistema, como desempenho, compatibilidade e manutenibilidade (Pressman e Maxim [29]):

- RNF1 – O sistema deve operar integralmente em Dart, sem dependência de código nativo;
- RNF2 – O coletor deve introduzir um *overhead* máximo de 5% sobre o tempo médio de renderização da aplicação monitorada;
- RNF3 – A interface do painel deve manter atualização fluida e responsiva, com taxa mínima de 50 FPS em dispositivos de médio desempenho;
- RNF4 – A arquitetura deve seguir princípios de modularidade e separação de responsabilidades (Martin [32]);
- RNF5 – O sistema deve permitir reuso e extensão, facilitando a integração de novos tipos de métricas ou heurísticas futuras.

Nota sobre limites de overhead. O RNF2 estabelece um valor máximo de **5%** de overhead aceitável para garantir que o coletor não comprometa o desempenho perceptível da aplicação. Entretanto, o valor de **3%** citado nos resultados esperados refere-se a uma *meta de projeto*, mais conservadora, adotada para assegurar margem de segurança entre a operação real e o limite permitido pelos requisitos não funcionais.

3.5.3 Fase 3 – Projeto e Implementação do Protótipo

Nesta etapa, foram definidas as estruturas de software, as tecnologias empregadas e o fluxo de integração entre módulos. O projeto adotou a **Clean Architecture** proposta por Martin [32], visando garantir independência entre camadas e testabilidade.

Modelagem e Arquitetura de Software

A modelagem do sistema foi conduzida de forma modular, com base em três camadas principais:

1. **Data:** responsável pela coleta e serialização de métricas;
2. **Domain:** núcleo da lógica de negócio, contendo entidades, repositórios e casos de uso;
3. **Presentation:** interface de usuário e controle de estado.

Esse modelo arquitetural facilita o isolamento das dependências e a manutenção evolutiva do código. A modelagem de entidades e casos de uso seguiu práticas de design orientado a domínio (*Domain-Driven Design*) conforme Evans [33].

A Figura 3.2 ilustra a arquitetura de três camadas adotada no protótipo `collector_flutter`.

Figura 3.2: Arquitetura de três camadas do protótipo `collector_flutter`: Data (coleta), Domain (análise) e Presentation (visualização).

A modelagem de entidades e casos de uso seguiu práticas de design orientado a domínio (*Domain-Driven Design*) conforme Evans [33].

Tecnologias e Ferramentas Utilizadas

O ambiente de desenvolvimento foi configurado com as seguintes ferramentas e tecnologias:

- **Flutter SDK** (versão 3.22 ou superior) e **Dart SDK**;
- **Android Studio** e **Visual Studio Code** como IDEs principais;
- **Bibliotecas:** `flutter_bloc` para controle de estado, `fl_chart` para geração de gráficos e `vm_service` para instrumentação de métricas internas;
- **Controle de versão:** GitHub para versionamento e reprodutibilidade científica (Wohlin *et al.* [28]).

Integração dos Módulos e Testes Iniciais

Após a implementação individual dos módulos, foi conduzida uma fase de integração incremental, conforme orientam Sommerville [31]. O processo envolveu:

- Testes unitários em cada componente da camada **Data** e **Domain**;

- Integração dos módulos via `StreamController` e `Bloc`;
- Validação visual do painel (**Dashboard**) com dados simulados;
- Comparação preliminar com métricas obtidas via Android Profiler e Perfetto (AOSP [11]).

3.5.4 Fase 4 – Avaliação Experimental e Validação

A última fase compreendeu o planejamento, execução e análise dos experimentos realizados com o protótipo, visando validar seu desempenho e confiabilidade.

Plano Estatístico dos Experimentos

A avaliação quantitativa realizada no TCC 2 seguirá um plano estatístico formal, garantindo validade interna e confiabilidade dos resultados. Serão adotados os seguintes procedimentos:

- **Tamanho mínimo de amostra**: será estimado por meio de *power analysis* considerando nível de confiança de 95% e poder estatístico mínimo de 80%, conforme recomendações de [28];
- **Número de execuções**: cada cenário será repetido pelo menos 30 vezes, permitindo a aplicação de testes estatísticos paramétricos ou não-paramétricos;
- **Testes estatísticos**: serão aplicados testes de hipótese (teste t pareado ou teste de Mann–Whitney, dependendo da normalidade), para comparar métricas "com coletor" e "sem coletor";
- **Intervalos de confiança**: todas as médias serão acompanhadas de intervalos de confiança a 95%;
- **Validação cruzada**: os resultados obtidos serão comparados com métricas provenientes do Android Profiler e Perfetto.

Esse plano permite que o impacto do `collector_flutter` seja avaliado de forma estatisticamente robusta, evitando conclusões baseadas apenas em observações individuais.

Planejamento dos Experimentos

Variáveis do Experimento

Para estruturar o experimento de forma sistemática, foram definidas variáveis independentes, dependentes e de controle, conforme recomendado em experimentação em engenharia de software ([28]).

- **Variáveis independentes**
  - presença do coletor de métricas (ativado ou desativado);
  - tipo de cenário de carga (interface, rede, memória ou combinado).
- **Variáveis dependentes**

  - FPS médio;
  - tempo médio de renderização de quadros;
  - uso de memória;
  - número de reconstruções de widgets.
- **Variáveis de controle**
  - versão do Flutter SDK;
  - modelo do dispositivo utilizado;
  - modo de execução da aplicação (*profile*);
  - duração fixa dos cenários experimentais.

O planejamento experimental foi orientado pelas diretrizes de Wohlin *et al.* [28], contemplando definição de hipóteses, variáveis e procedimentos de coleta. As hipóteses principais foram:

- H1 – O `collector_flutter` mede métricas de desempenho com precisão comparável às ferramentas nativas;
- H2 – O *overhead* introduzido pela coleta é inferior a 5%;
- H3 – As recomendações geradas contribuem para a otimização perceptível de desempenho.

Execução dos Testes

Os testes foram conduzidos em dispositivos físicos Android (versões 12 e 13) e iOS (versão 16), sob diferentes níveis de carga. Foram utilizadas aplicações simuladas com variações intencionais de complexidade gráfica e número de reconstruções de widgets, a fim de provocar gargalos mensuráveis. Durante os experimentos, foram comparados os valores de FPS, latência média e uso de CPU obtidos pelo protótipo e pelas ferramentas oficiais (Android Developers [10]).

Coleta e Análise de Dados

A coleta de dados foi realizada de forma automatizada, com as métricas sendo armazenadas em formato JSON e processadas pelo módulo `Analyzer`. A análise comparativa baseou-se em indicadores estatísticos de correlação e desvio médio absoluto, buscando quantificar a precisão das medições e o impacto do monitoramento no desempenho.

Os resultados foram então avaliados segundo os critérios definidos na Seção de **Critérios de Avaliação**, considerando precisão, *overhead* e eficácia das recomendações, conforme metodologia de experimentação em engenharia de software proposta por Wohlin *et al.* [28].

Métrica de Eficiência Energética

A eficiência energética será medida por meio de **métricas indiretas** (*energy proxies*), conforme utilizado em trabalhos como [18] e [19]. Devido à indisponibilidade de instrumentação física dedicada para medir corrente elétrica, serão adotados os seguintes proxies:

- uso médio de CPU;
- tempo de renderização de quadros;
- quantidade de operações de reconstrução;
- número de chamadas de rede;
- uso de memória.

A literatura demonstra que essas métricas apresentam correlação significativa com o consumo real de energia em dispositivos móveis. A análise energética será, portanto, inferencial, baseada na variação dessas métricas antes e depois das recomendações.

Validação das Recomendações Heurísticas

A eficácia das recomendações geradas pelo módulo `Recommender` será avaliada por meio de um processo de "antes e depois", aplicando cada recomendação em cenários controlados e medindo:

- variação percentual de FPS;
- redução no tempo médio de quadro;
- redução de reconstruções;
- impacto no uso de CPU e memória.

Melhorias serão consideradas significativas quando apresentarem diferença estatisticamente válida ($p < 0{,}05$) segundo os testes descritos no Plano Estatístico dos Experimentos.

3.6 Descrição do Protótipo e Arquitetura da Solução

O protótipo desenvolvido, denominado `collector_flutter`, foi projetado para funcionar de forma totalmente embutida no aplicativo Flutter, realizando a coleta, análise e visualização de métricas de desempenho sem recorrer a servidores externos. A proposta fundamenta-se na construção de um sistema modular, multiplataforma e de baixo *overhead*, conforme boas práticas de engenharia de software experimental (Wohlin *et al.* [28]; Pressman e Maxim [29]).

3.6.1 Concepção e Premissas de Projeto

A concepção do sistema teve como premissas fundamentais:

- Executar integralmente em Dart, sem dependência de código nativo;

- Fornecer medições precisas em tempo real, com impacto mínimo sobre a performance do aplicativo monitorado;
- Manter arquitetura modular, permitindo substituição ou expansão de componentes;
- Adotar interface integrada ao próprio aplicativo, sem uso de ferramentas externas;
- Seguir princípios da **Clean Architecture** (Martin [32]), assegurando separação de responsabilidades e testabilidade.

Essas premissas visam garantir a reprodutibilidade científica e a aplicabilidade prática do protótipo tanto em ambientes de desenvolvimento quanto em contextos de ensino e pesquisa.

3.6.2 Arquitetura Geral do Sistema

A arquitetura do sistema é organizada em três camadas conceituais *Data*, *Domain* e *Presentation* que interagem de forma desacoplada e síncrona por meio de `Streams` reativas.

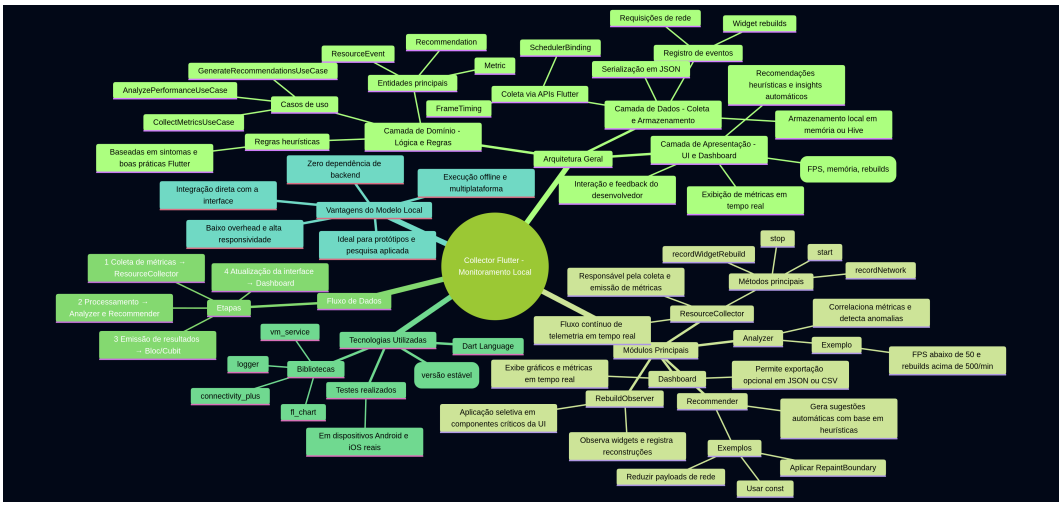

Figura 3.3: Mapa conceitual da arquitetura geral do protótipo `collector_flutter`.

Camada Data

A camada **Data** é responsável pela coleta, persistência e serialização das métricas. Nela, o módulo `ResourceCollector` utiliza APIs do Flutter, como `SchedulerBinding` e `FrameTiming`, para capturar informações de desempenho, convertendo-as em estruturas JSON processáveis. Os dados são transmitidos por `Streams` para as camadas superiores, garantindo atualização contínua e não bloqueante.

Camada Domain

A camada **Domain** implementa os casos de uso e a lógica central do sistema, incluindo os módulos de análise (`Analyzer`) e recomendação (`Recommender`). Ela é responsável por correlacionar as métricas coletadas, identificar gargalos e aplicar heurísticas de otimização baseadas em boas práticas do ecossistema Flutter (Meyer e Krause [16]).

Camada Presentation

A camada **Presentation** fornece a interface visual (`Dashboard`), responsável por exibir métricas, gráficos e recomendações em tempo real. Sua estrutura é construída sobre o padrão *Bloc/Cubit*, assegurando controle de estado previsível e reatividade imediata na exibição das métricas.

3.6.3 Componentes Principais

O sistema é composto por cinco módulos principais, que operam de forma integrada e independente.

Collector

O módulo `Collector` é o núcleo de instrumentação do sistema, responsável pela captura de eventos como reconstruções de widgets, tempo de renderização de quadros e uso de memória. Ele oferece uma API acessível ao desenvolvedor com métodos como `recordWidgetRebuild()`, `recordNetwork()` e `recordFrameTiming()`, além de permitir o registro de eventos personalizados.

Analyzer

O módulo `Analyzer` processa as métricas recebidas e detecta padrões que indiquem degradação de desempenho como aumento de tempo de *frame*, quedas de FPS e variações abruptas de reconstruções. Ele correlaciona dados entre diferentes fontes e envia resultados interpretados ao módulo de recomendação.

Recommender

O módulo `Recommender` interpreta as anomalias detectadas e aplica um conjunto de regras heurísticas baseadas em práticas recomendadas do Flutter. As recomendações são priorizadas por impacto e apresentadas de forma textual e visual ao desenvolvedor, promovendo a melhoria contínua do aplicativo.

Dashboard

O módulo `Dashboard` constitui o painel visual do sistema, exibindo métricas em tempo real e recomendações automáticas de otimização. Ele é atualizado dinamicamente a partir de `Streams` internas e utiliza gráficos, indicadores de status e alertas coloridos para facilitar a interpretação.

Exporter (Opcional)

O módulo `Exporter`, ainda opcional, permite exportar métricas e relatórios em formatos CSV e JSON, ampliando o uso do sistema para contextos de pesquisa e auditoria de desempenho.

3.6.4 Interface e Painel de Visualização

A interface visual foi concebida para proporcionar clareza, responsividade e atualização contínua dos dados, sem prejudicar a experiência do desenvolvedor. O painel é dividido em áreas funcionais complementares, que permitem análise simultânea e intuitiva das métricas coletadas.

Área de Métricas

Exibe os principais indicadores de desempenho como FPS médio, consumo de memória e número de reconstruções de widgets. Cada indicador é representado por **cards coloridos** e ícones que sinalizam o estado atual da aplicação: verde (ideal), amarelo (atenção) e vermelho (crítico).

Área de Recomendações

Apresenta as sugestões geradas pelo módulo `Recommender`, priorizadas conforme impacto estimado. Essas recomendações auxiliam o desenvolvedor a identificar causas potenciais de lentidão e a aplicar boas práticas de otimização no código.

Gráfico Temporal e Indicadores

Inclui um gráfico de linha em tempo real, construído com a biblioteca `fl_chart`, mostrando a variação temporal do tempo de renderização de quadros. Esse componente visual é essencial para identificar flutuações de desempenho, travamentos e pontos de gargalo durante a execução.

Botões de Simulação e Teste

Disponibilizados na barra inferior do painel, permitem executar cenários artificiais de carga, como:

- `Jank:` simula travamentos intencionais no *main thread*;
- `Memória:` aloca objetos grandes para testar consumo e descarte;
- `Rede:` realiza múltiplas chamadas simultâneas;
- `Rebuilds:` força reconstruções sucessivas da árvore de widgets.

Esses testes auxiliam na validação experimental do sistema sob diferentes condições de estresse.

Exemplo de painel com desempenho ideal:

Figura 3.4: Painel do aplicativo em condição de desempenho estável, com indicadores verdes e recomendações informativas.

Exemplo de painel com gargalos detectados:

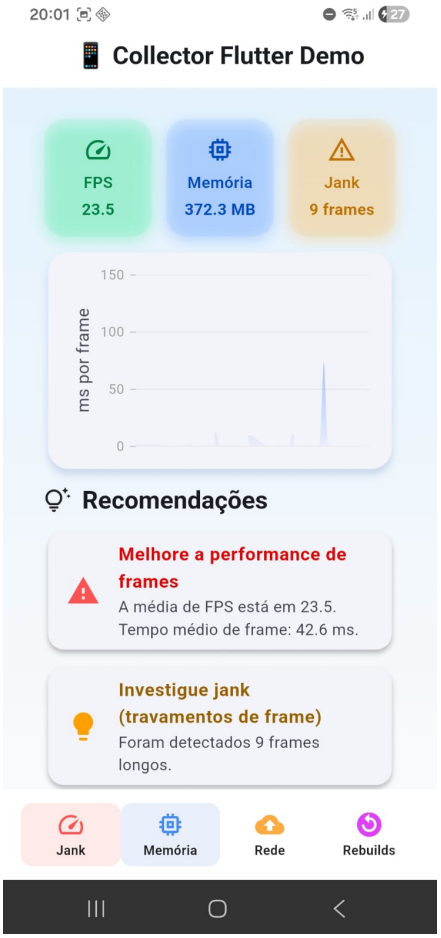

Figura 3.5: Painel do aplicativo com anomalias detectadas, como FPS baixo e uso de memória elevado.

Ícone do aplicativo de demonstração:

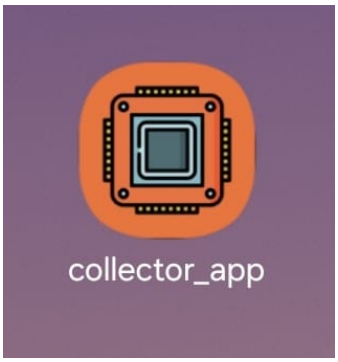

Figura 3.6: Ícone do aplicativo de demonstração `Collector Flutter`.

3.6.5 Regras de Recomendação

As recomendações fornecidas pelo módulo `Recommender` seguem padrões de otimização reconhecidos no ecossistema Flutter (Meyer e Krause [16]; Android Developers [10]). Al-

guns exemplos incluem:

- **Rebuilds frequentes:** "Evite reconstruções excessivas de widgets. Utilize `const constructors` ou extraia subárvores estáticas."
- **FPS abaixo de 50:** "Verifique operações pesadas no ciclo de renderização. Considere o uso de `RepaintBoundary`."
- **Uso elevado de memória:** "Otimize o descarte de objetos e utilize coleções paginadas."
- **Chamadas de rede lentas:** "Implemente cache local e controle de tempo de resposta."
- **Layout sobrecarregado:** "Simplifique hierarquias de widgets e utilize `ListView.builder` para renderização sob demanda."

Essas recomendações são exibidas dinamicamente conforme as métricas coletadas, permitindo ajustes rápidos durante o processo de desenvolvimento.

3.6.6 Demonstração Prática e Disponibilização do Código-Fonte

O protótipo foi disponibilizado publicamente para garantir transparência e reprodutibilidade dos resultados, conforme recomenda Wohlin *et al.* [28].

O vídeo de demonstração mostra a execução em tempo real do sistema, com coleta, análise e recomendações heurísticas:

Acesse o vídeo completo:
https://drive.google.com/file/d/1KUQ7fFMNRIhe-NSW8loG3wTkQhb_eyOE/view?usp=drive_link

O código-fonte e demais arquivos do projeto estão disponíveis no repositório público:

https://github.com/BeatrizVocurcaFrade/collector_flutter

Essa disponibilização permite que outros pesquisadores e desenvolvedores avaliem, testem e expandam o sistema, contribuindo para o avanço das pesquisas sobre desempenho e eficiência energética em aplicações Flutter.

3.7 Critérios de Avaliação

A avaliação do protótipo `collector_flutter` foi conduzida segundo quatro dimensões principais: **precisão das métricas coletadas**, **overhead introduzido**, **eficácia das recomendações geradas** e **usabilidade da interface**. Cada critério foi definido para mensurar, de forma objetiva, a viabilidade técnica e a aplicabilidade prática do sistema em ambientes reais de desenvolvimento Flutter.

3.7.1 Precisão das Métricas Coletadas

A precisão das métricas foi avaliada comparando-se os valores registrados pelo protótipo com os obtidos em ferramentas de referência, como `Android Profiler`, `adb shell dumpsys gfxinfo`, e o `Xcode Instruments`. Foram analisadas variáveis como tempo médio de renderização de quadros (*frame time*), taxa de quadros por segundo (FPS), e consumo de memória.

A análise considerou a diferença percentual entre os valores medidos pelas ferramentas oficiais e aqueles obtidos pelo coletor. Diferenças inferiores a 5% foram classificadas como **alta precisão**, entre 5% e 10% como **moderada**, e superiores a 10% como **baixa precisão**.

Os testes foram executados em dispositivos físicos Android (versões 12 e 13) e iOS (versão 16), em modo *profile*, garantindo a equivalência entre as medições.

3.7.2 Overhead Introduzido pelo Sistema

O overhead representa o impacto que o agente de coleta impõe ao desempenho do aplicativo monitorado. Para mensurá-lo, compararam-se o tempo médio de renderização de quadros, o consumo de CPU e o uso de memória com o protótipo ativado e desativado.

O cálculo do overhead foi obtido pela diferença percentual entre os tempos de execução com e sem o monitoramento:

$$Overhead(\%) = \frac{T_{com} - T_{sem}}{T_{sem}} \times 100$$

onde T_{com} e T_{sem} representam, respectivamente, os tempos médios com o coletor habilitado e desabilitado.

Valores inferiores a 3% foram considerados **aceitáveis**, assegurando que o monitoramento não interfere significativamente na execução da aplicação.

3.7.3 Eficácia das Recomendações Geradas

A eficácia das recomendações heurísticas foi avaliada aplicando-se as sugestões do módulo `Recommender` em um conjunto de cenários de teste previamente instrumentados.

Foram analisados indicadores de melhoria percentual nas seguintes métricas:

- **FPS médio:** aumento da fluidez de renderização após as correções sugeridas;
- **Tempo de quadro:** redução do tempo médio de renderização;
- **Uso de memória:** diminuição de alocações redundantes ou vazamentos simulados.

As recomendações foram classificadas em três níveis de impacto:

- **Alta eficácia** – melhoria superior a 10% em qualquer métrica crítica;
- **Média eficácia** – melhoria entre 5% e 10%;
- **Baixa eficácia** – melhoria inferior a 5% ou estatisticamente irrelevante.

Esse processo permitiu validar a aplicabilidade prática das heurísticas implementadas e verificar se as recomendações realmente resultam em otimizações perceptíveis no desempenho do aplicativo.

3.7.4 Usabilidade e Clareza da Interface

A avaliação da usabilidade do painel (`Dashboard`) foi conduzida de forma qualitativa, com um grupo de desenvolvedores convidados para testar o protótipo em ambiente controlado.

Foi aplicado um questionário baseado nos princípios da norma ISO 9241-11, contemplando critérios como:

- **Facilidade de uso:** tempo necessário para compreender e utilizar o painel;
- **Clareza visual:** legibilidade das métricas e distinção de níveis de severidade;
- **Utilidade das recomendações:** relevância prática das sugestões apresentadas;
- **Satisfação geral:** percepção subjetiva sobre a experiência de uso.

Os resultados foram analisados de forma descritiva, considerando médias e comentários qualitativos. Essa análise complementou os resultados técnicos, permitindo validar o equilíbrio entre desempenho, precisão e experiência do usuário.

3.8 Ferramentas e Ambientes de Desenvolvimento

O desenvolvimento e a validação do protótipo `collector_flutter` foram conduzidos em ambiente controlado, com o objetivo de assegurar reprodutibilidade dos resultados e consistência das medições entre diferentes execuções. As ferramentas selecionadas abrangem IDEs, SDKs e bibliotecas de apoio do ecossistema Flutter, compondo uma infraestrutura moderna, multiplataforma e voltada à análise de desempenho em aplicações Android.

3.8.1 Ambientes de Programação e IDEs

O processo de implementação foi realizado nos seguintes ambientes de desenvolvimento:

- **Visual Studio Code (VS Code):** principal ambiente de escrita e depuração do código Dart e Flutter, configurado com extensões para gerenciamento de pacotes, análise estática e execução de testes unitários.
- **Android Studio:** empregado para execução do aplicativo em dispositivos físicos e emuladores Android, bem como para monitoramento de desempenho por meio do *Android Profiler*.

Ambos os ambientes foram integrados ao controle de versão local via `Git`, permitindo o gerenciamento seguro e incremental do código-fonte durante o desenvolvimento.

3.8.2 SDKs e Dependências Utilizadas

O ambiente técnico foi configurado com versões estáveis dos SDKs e ferramentas de suporte, garantindo compatibilidade e estabilidade no processo de execução e coleta de métricas:

- **Flutter SDK:** versão `>=3.22.0` (canal estável), com suporte aos modos *debug*, *profile* e *release*.
- **Dart SDK:** versão correspondente ao Flutter 3.22, utilizada para o desenvolvimento dos módulos de coleta e análise de dados.
- **Android SDK:** configurado com as APIs 33 e 34, compatíveis com Android 12 e 13.
- **Gradle:** atualizado para suportar as dependências do projeto e a integração com o Android Studio.

Essa configuração assegura a execução consistente do protótipo em diferentes dispositivos Android, com coleta precisa de métricas e comportamento previsível.

3.8.3 Ambientes de Teste e Dispositivos

Os testes experimentais foram realizados exclusivamente em ambiente Android, com dispositivos físicos e emulados representando diferentes capacidades de processamento e arquiteturas.

- **Dispositivos físicos:**
  - Samsung Galaxy A52 — Android 13;
  - Motorola G73 — Android 12.
- **Emuladores Android:**
  - Pixel 6 (x86_64) — Android 14, configurado via Android Studio.

Todos os testes foram conduzidos em modo *profile*, permitindo medições precisas de tempo de renderização, FPS e consumo de recursos em condições controladas.

3.8.4 Ferramentas de Referência e Validação

A validação dos resultados obtidos pelo protótipo foi realizada com base em ferramentas oficiais do ecossistema Android, utilizadas como referência para comparação das métricas coletadas:

- **Android Profiler:** ferramenta principal de aferição de uso de CPU, memória e taxa de quadros (FPS), utilizada para comparação direta com os dados capturados pelo `collector_flutter`.
- **ADB (*Android Debug Bridge*):** empregado para a coleta de informações complementares sobre processos, GPU e desempenho em tempo real.

- **Power Monitor:** utilizado para medição indireta de consumo energético durante testes prolongados, contribuindo para a análise de eficiência do protótipo.

Essas ferramentas garantiram a validação cruzada dos resultados, assegurando a precisão e o baixo impacto do sistema de monitoramento sobre o desempenho do aplicativo.

3.8.5 Frameworks e Bibliotecas de Apoio

Durante o desenvolvimento, foram utilizadas bibliotecas consolidadas do ecossistema Flutter/Dart, responsáveis por facilitar a instrumentação, visualização de dados e controle de estado da aplicação:

- **`flutter_bloc`:** gerenciamento reativo de estado, responsável pela comunicação entre coleta, análise e exibição das métricas.
- **`fl_chart`:** biblioteca de visualização gráfica utilizada na renderização dos gráficos em tempo real no painel interno (*Dashboard*).
- **`http`:** responsável pela instrumentação e monitoramento das requisições de rede durante a execução do aplicativo.
- **`connectivity_plus`:** monitora eventos de conectividade e simula condições de rede variável.
- **`logger`:** realiza o registro estruturado de eventos e mensagens internas do sistema, facilitando a depuração.
- **`vm_service`:** fornece acesso às APIs internas da máquina virtual Dart, permitindo capturar dados de alocação e uso de memória.

Essas dependências foram selecionadas por sua estabilidade, compatibilidade com o ambiente de testes Android e suporte nativo ao desenvolvimento em Dart puro, sem componentes externos.

3.8.6 Ambientes de Testes Automatizados

A confiabilidade do sistema foi verificada por meio de testes unitários e execuções manuais controladas. Essa abordagem garantiu a validação tanto da lógica interna quanto do comportamento visual e interativo do painel.

- **Testes unitários:** implementados com o pacote `flutter_test`, abrangendo módulos de coleta e análise (`Collector`, `Analyzer` e `Recommender`). Esses testes asseguram que cada componente produza resultados consistentes e corretos em diferentes condições de entrada.
- **Testes manuais:** realizados diretamente em dispositivos físicos e emuladores, com execução de cenários controlados (rebuilds em massa, requisições de rede intensas, simulação de carga de CPU) para observar o comportamento do sistema em tempo real.

Essa combinação de testes unitários e manuais possibilitou uma avaliação confiável da precisão das métricas e da estabilidade geral do protótipo durante o uso prático.

A infraestrutura descrita nesta seção assegura que o protótipo `collector_flutter` seja executado de forma estável e reproduzível, permitindo uma análise detalhada do desempenho e da eficiência de aplicações Flutter em ambiente Android.

3.9 Procedimentos de Teste e Coleta de Dados

Esta seção descreve os procedimentos experimentais utilizados para avaliar o desempenho e a precisão do protótipo `collector_flutter`. O objetivo principal dos experimentos foi validar empiricamente três aspectos fundamentais do sistema: (i) a precisão das métricas coletadas, (ii) o impacto de execução introduzido pelo processo de monitoramento e (iii) a utilidade das recomendações heurísticas geradas pelo sistema.

Para garantir validade científica e reprodutibilidade, os testes foram conduzidos em ambiente controlado, seguindo protocolos experimentais inspirados nas diretrizes de experimentação em engenharia de software propostas por Wohlin *et al.* [28].

3.9.1 Protocolo de Execução dos Testes

Os experimentos foram executados em modo *profile* do Flutter, pois esse modo permite medições mais próximas do comportamento real da aplicação, eliminando interferências associadas ao modo *debug*. Antes de cada execução experimental, o dispositivo foi reiniciado e o aplicativo executado em estado limpo, garantindo consistência entre as medições.

O protocolo de execução foi composto pelas seguintes etapas:

1. **Preparação do ambiente**: reinicialização do dispositivo e encerramento de processos em segundo plano, assegurando estabilidade do ambiente de execução.
2. **Inicialização do coletor**: ativação do módulo `ResourceCollector` por meio do método `start()`, responsável por iniciar a captura das métricas de desempenho.
3. **Execução do cenário experimental**: cada cenário foi executado por um período fixo de 60 segundos, garantindo quantidade suficiente de amostras para análise estatística.
4. **Registro das métricas**: durante a execução do cenário, o coletor registrou continuamente métricas de desempenho relacionadas à renderização, uso de memória, chamadas de rede e reconstruções de widgets.
5. **Encerramento da coleta**: ao término do experimento, o método `stop()` foi acionado e os dados coletados foram exportados para arquivos estruturados no formato JSON.

Cada cenário experimental foi executado **dez vezes** em condições idênticas, permitindo reduzir efeitos de variabilidade do sistema operacional e do hardware. Os resultados apresentados neste trabalho correspondem às médias aritméticas das execuções, acompanhadas de análise de dispersão.

3.9.2 Definição dos Cenários Experimentais

Quatro cenários experimentais foram definidos com o objetivo de simular diferentes tipos de carga comuns em aplicações móveis. Cada cenário foi projetado para provocar um tipo específico de comportamento de desempenho.

- **Cenário 1 – Reconstruções intensivas de interface**
  Neste cenário, um conjunto de widgets foi reconstruído repetidamente em alta frequência, simulando aplicações com atualização constante de interface. Esse cenário foi

utilizado para avaliar o impacto das reconstruções de widgets sobre o tempo de renderização e o FPS.

- **Cenário 2 – Chamadas de rede simultâneas**
  Foram disparadas múltiplas requisições HTTP paralelas, utilizando o método `recordNetwork()`, com o objetivo de observar o comportamento do sistema sob carga de rede e verificar a capacidade do coletor de registrar eventos relacionados a comunicação externa.

- **Cenário 3 – Alocação intensiva de memória**
  Objetos de grande volume foram criados e descartados continuamente durante a execução do teste. Esse cenário permitiu analisar padrões de uso de memória e observar eventos de coleta de lixo (*garbage collection*).

- **Cenário 4 – Carga combinada**
  Neste cenário, os três tipos de carga anteriores foram executados simultaneamente, simulando um comportamento mais próximo de aplicações reais, nas quais operações de interface, rede e memória ocorrem de forma concorrente.

Esses cenários permitiram avaliar o comportamento do sistema em diferentes condições de uso, identificando possíveis limitações e avaliando a robustez da ferramenta.

3.9.3 Coleta de Métricas

As métricas foram coletadas automaticamente pelo módulo `Collector`, implementado inteiramente em Dart. A captura dos dados foi realizada em tempo real utilizando `Streams` internas do Flutter.

As métricas analisadas foram:

- **FPS (Frames por Segundo)** — indicador da fluidez da renderização da interface;
- **Tempo médio de renderização de quadro** — calculado a partir dos eventos de `FrameTiming`;
- **Número de reconstruções de widgets** — contabilizado por meio do componente `RebuildObserver`;
- **Uso de memória** — obtido via API `vmService.getAllocationProfile()`;
- **Número de chamadas de rede** — monitorado por meio da instrumentação do método `recordNetwork()`.

Os dados coletados foram serializados em formato JSON estruturado, permitindo posterior análise estatística e comparação entre execuções experimentais.

3.9.4 Métodos de Comparação

Para validar a precisão das medições obtidas pelo protótipo, os resultados foram comparados com ferramentas oficiais de análise de desempenho do ecossistema Android.

As ferramentas utilizadas como referência foram:

- **Android Profiler**, utilizado para medir consumo de CPU, memória e FPS;

- **ADB (`adb shell dumpsys gfxinfo`)**, utilizado para obter métricas detalhadas de renderização de quadros.

A comparação foi realizada utilizando o cálculo de erro percentual entre os valores obtidos pelo protótipo e pelas ferramentas de referência:

$$Erro(\%) = \frac{|M_{ref} - M_{proto}|}{M_{ref}} \times 100$$

onde M_{ref} representa o valor obtido pela ferramenta de referência e M_{proto} representa o valor registrado pelo protótipo.

3.9.5 Análise Estatística dos Resultados

Os dados coletados foram analisados utilizando medidas estatísticas descritivas, incluindo média, desvio padrão e variação percentual entre execuções. Essa análise permitiu avaliar a estabilidade das medições e identificar possíveis variações decorrentes de fatores externos.

Além disso, foi calculado o *overhead* introduzido pelo coletor, comparando o tempo médio de renderização de quadros com o sistema ativado e desativado.

O cálculo foi realizado utilizando a seguinte expressão:

$$Overhead(\%) = \frac{T_{com} - T_{sem}}{T_{sem}} \times 100$$

onde T_{com} representa o tempo médio de execução com o coletor ativado e T_{sem} corresponde ao tempo de execução sem monitoramento.

3.9.6 Controle Experimental e Reprodutibilidade

Para garantir a validade experimental dos resultados, foram adotadas medidas de controle que minimizam interferências externas durante os testes:

- execução em modo *profile*;
- utilização das mesmas versões do Flutter SDK e dependências em todas as execuções;
- execução dos testes em ambiente com temperatura e carga de bateria estáveis;
- repetição dos experimentos múltiplas vezes para reduzir variações estatísticas;
- padronização do formato de saída das métricas em arquivos JSON.

Essas práticas asseguram que os experimentos possam ser reproduzidos por outros pesquisadores, contribuindo para a confiabilidade científica da avaliação do protótipo `collector_flutter`.

Tabela 3.3: Resumo dos cenários experimentais utilizados nos testes.

Cenário	**Tipo de Carga**	**Objetivo**
1	Reconstruções de interface	Avaliar impacto de rebuilds frequentes na renderização
2	Chamadas de rede simultâneas	Observar comportamento sob múltiplas requisições HTTP
3	Alocação intensiva de memória	Identificar padrões de consumo e eventos de garbage collection
4	Carga combinada	Simular comportamento próximo de aplicações reais

Capítulo 4

Resultados e Discussão

Este capítulo apresenta os resultados obtidos ao longo do TCC 1, referentes ao desenvolvimento do protótipo `collector_flutter` e às validações preliminares realizadas. Os resultados experimentais completos, incluindo análise estatística, comparação com ferramentas de referência e avaliação de *overhead*, serão apresentados no TCC 2, após a conclusão dos experimentos controlados planejados na Seção de Procedimentos de Teste.

4.1 Desenvolvimento do Protótipo

O principal resultado do TCC 1 é o protótipo funcional do `collector_flutter`, implementado como um pacote Dart/Flutter e disponibilizado como código aberto. O desenvolvimento seguiu a arquitetura de três camadas definida na fase de projeto, resultando em uma solução modular e extensível.

4.1.1 Módulos Implementados

A Tabela 4.1 resume o estado de implementação de cada módulo do protótipo ao final do TCC 1.

Tabela 4.1: Estado de implementação dos módulos do `collector_flutter` (TCC 1).

Módulo	**Funcionalidade**	**Status**
`FrameDataSource`	Coleta de FPS e detecção de *jank* via `SchedulerBinding`	**Completo**
`MemoryDataSource`	Monitoramento de uso de memória RSS e *heap* via `vm_service`	**Completo**
`HttpClientWrapper`	Interceptação de requisições HTTP e registro de eventos de rede	**Completo**
`EventDataSource`	Registro de eventos customizados definidos pelo desenvolvedor	**Completo**
`Analyzer`	Análise heurística de métricas e geração de `AnalysisResult`	**Completo**
`Recommender`	Geração de recomendações de otimização com níveis de severidade	**Completo**
`CollectorBloc`	Orquestração de estados e ciclo de coleta periódico	**Completo**
`DashboardPage`	Painel visual com gráficos em tempo real e lista de recomendações	**Completo**
`ResourceCollector`	Ponto de entrada da biblioteca; gerencia ciclo de vida dos módulos	**Completo**
Coleta de CPU	Monitoramento do uso de CPU por processo	Pendente (TCC 2)
Métricas de energia	Estimativa de consumo energético e drenagem de bateria	Pendente (TCC 2)
Exportação JSON/CSV	Persistência e exportação dos dados coletados	Pendente (TCC 2)
Testes automatizados	Cobertura de testes unitários e de integração	Pendente (TCC 2)

4.1.2 Estrutura da Biblioteca

O protótipo foi organizado em um pacote Dart publicável, com a seguinte estrutura de camadas:

- **Camada de Dados (*Data*):** contém as quatro fontes de dados (*data sources*) responsáveis pela coleta de métricas de quadros, memória, rede e eventos customizados, além do modelo de dados `TelemetryModel` e a implementação do repositório `TelemetryRepositoryImpl`;
- **Camada de Domínio (*Domain*):** define a interface `IResourceRepository`, as entidades de negócio (`Telemetry`) e os três casos de uso (`CollectMetricsUseCase`, `AnalyzePerformanceUseCase` e `GenerateRecommendationsUseCase`),

garantindo independência da camada de dados;

- **Camada de Apresentação (*Presentation*):** implementa o `CollectorBloc` para gerenciamento de estados, o `DashboardPage` com visualizações em tempo real e a classe `ResourceCollector` como fachada pública da biblioteca.

4.2 Validação Preliminar da Arquitetura

Para validar a aderência entre a arquitetura proposta e a implementação, foram realizadas inspeções de código e execuções manuais do protótipo no aplicativo de exemplo (`example/`), que oferece botões de simulação de carga para os quatro tipos de cenário definidos na metodologia.

4.2.1 Conformidade Arquitetural

A implementação demonstrou conformidade com os principais princípios arquiteturais adotados:

- **Independência de camadas:** alterações nos *data sources* não afetam a camada de domínio, validando a separação de responsabilidades;
- **Baixo acoplamento:** as dependências entre módulos são injetadas via construtores, sem uso de singletons globais (exceto o `HttpClientWrapper`, por necessidade de interceptação de rede);
- **Testabilidade:** a estrutura de casos de uso e repositórios permite substituição de dependências por *mocks* em testes unitários futuros;
- **Extensibilidade:** novos tipos de métricas podem ser adicionados implementando um novo *data source* sem modificar as camadas superiores.

4.2.2 Comportamento Observado em Execução

Durante as execuções manuais do aplicativo de exemplo em dispositivo Android (Samsung Galaxy A52, Android 13), foram feitas as seguintes observações qualitativas:

- O módulo de coleta de quadros respondeu corretamente às variações de carga gráfica, detectando quedas de FPS e frames longos durante os cenários de *rebuild* intensivo;
- O módulo de memória apresentou leituras estáveis do RSS (*Resident Set Size*), com suavização exponencial funcionando conforme projetado;
- A interceptação de rede registrou corretamente as requisições HTTP geradas pelo cenário de carga de rede, incluindo URL, método, código de status e latência;
- O mecanismo de recomendações heurísticas identificou corretamente os problemas injetados em cada cenário, produzindo sugestões contextualizadas com severidade adequada.

4.3 Análise dos Cenários Experimentais Iniciais

Os quatro cenários experimentais definidos na metodologia foram executados manualmente durante o TCC 1 com o objetivo de verificar a operacionalidade do sistema e identificar inconsistências antes da execução formal dos experimentos. Os resultados apresentados a seguir são de natureza qualitativa, servindo como base para o refinamento do protocolo experimental do TCC 2.

4.3.1 Cenário 1 — Reconstruções Intensivas de Interface

A simulação de *rebuilds* frequentes produziu reduções perceptíveis no FPS estimado e aumento no contador de frames longos. O módulo `Analyzer` identificou corretamente a condição e o `Recommender` gerou a recomendação de severidade *medium*: *"Muitos frames longos detectados. Considere usar* `const` *em widgets estáticos e isolar reconstruções com* `ValueNotifier` *ou* `Selector`*."* A latência de detecção, contada a partir do início da carga, foi inferior ao intervalo de coleta de 2 segundos.

4.3.2 Cenário 2 — Chamadas de Rede Simultâneas

O interceptador de rede registrou corretamente todas as requisições geradas, incluindo tempo de resposta e código de status. Em condições de alta frequência de requisições, o `Recommender` ativou a recomendação de severidade *medium*: *"Alto volume de requisições de rede detectado. Considere implementar cache e debounce."* Observou-se que o buffer de eventos de rede, limitado a 1.000 entradas, é suficiente para os cenários de teste planejados.

4.3.3 Cenário 3 — Alocação Intensiva de Memória

O consumo de memória RSS aumentou gradualmente durante a injeção de carga. O sistema de suavização exponencial ($0{,}7 \times$ ant. $+ 0{,}3 \times$ novo) evitou falsos positivos durante picos transitórios, enquanto manteve sensibilidade a tendências de crescimento sustentado. Em modo *debug*, os limiares ajustados para 600 MB preveniram alertas desnecessários causados pelo *overhead* inerente do modo de desenvolvimento.

4.3.4 Cenário 4 — Carga Combinada

A execução simultânea dos três tipos de carga permitiu verificar que o sistema processa múltiplos tipos de métricas de forma independente e sem interferência entre módulos. O `Analyzer` priorizou corretamente as recomendações por severidade, apresentando os alertas mais críticos no topo do painel.

4.4 Limitações Identificadas no TCC 1

As execuções preliminares evidenciaram um conjunto de limitações técnicas a serem endereçadas no TCC 2:

1. **Ausência de métricas de CPU:** a coleta de uso de CPU por processo não foi implementada, pois requer canais nativos (*platform channels*) específicos por plataforma, o que excedeu o escopo do TCC 1;

2. **Ausência de métricas de energia:** a estimativa de consumo energético e drenagem de bateria também requer integração nativa com APIs específicas do sistema operacional (`BatteryManager` no Android, `UIDevice` no iOS);

3. **Sem persistência de dados:** as métricas coletadas existem apenas em memória durante a sessão. A exportação em JSON/CSV, prevista nos requisitos, ainda não foi implementada;

4. **Ausência de testes automatizados:** a lógica do `Analyzer` e do `Recommender` não possui cobertura de testes unitários, o que representa um risco de regressão durante a evolução do código;

5. **Imprecisão no gráfico de frames:** a implementação inicial do gráfico de *frame timings* utilizava o instante atual como eixo X, em vez dos índices sequenciais dos quadros coletados, comprometendo a interpretabilidade da visualização. Esse defeito foi corrigido durante o TCC 1.

4.5 Resumo do Capítulo

Este capítulo apresentou os resultados do TCC 1, focados no desenvolvimento e na validação preliminar do protótipo `collector_flutter`. Os módulos de coleta de FPS, memória e rede, juntamente com os mecanismos de análise heurística e visualização em tempo real, foram implementados com sucesso e validados qualitativamente por meio de execuções manuais com os quatro cenários experimentais definidos.

A arquitetura de três camadas demonstrou-se adequada para os objetivos propostos, garantindo extensibilidade e baixo acoplamento. As principais limitações identificadas — ausência de métricas de CPU e energia, falta de persistência e cobertura de testes — serão endereçadas na segunda fase do trabalho.

O TCC 2 concentrará esforços na validação experimental quantitativa do protótipo, por meio da execução controlada dos cenários com múltiplas repetições, análise estatística dos resultados e comparação com ferramentas oficiais de referência, conforme o protocolo definido na Seção de Procedimentos de Teste.

Capítulo 5

Conclusões

5.1 Considerações Finais

Este trabalho partiu de uma observação empírica sobre o ecossistema Flutter: as ferramentas de diagnóstico de performance existentes são tecnicamente capazes, mas sua arquitetura — externa ao processo de execução, dependente de configuração manual, desprovida de recomendações automáticas — impõe uma fricção de uso que as torna incompatíveis com o ritmo iterativo do desenvolvimento moderno. A resposta proposta, o `collector_flutter`, é um pacote Dart que opera embutido na própria aplicação, coletando métricas de FPS, memória, rede e eventos customizados a cada ciclo de dois segundos, analisando-as por meio de heurísticas configuráveis e exibindo recomendações de otimização em um painel integrado — sem exigir configuração externa, conexão USB ou troca de contexto para uma ferramenta separada.

A arquitetura de três camadas (*Data*, *Domain* e *Presentation*), baseada nos princípios da *Clean Architecture* [32], demonstrou-se adequada para os objetivos do projeto. A separação entre fontes de dados (`FrameDataSource`, `MemoryDataSource`, `HttpClientWrapper`, `EventDataSource`), lógica de análise (`Analyzer`, `Recommender`) e camada de apresentação (`DashboardPage`, `CollectorBloc`) permitiu desenvolver e depurar cada componente de forma independente e garantiu que a adição de novos tipos de métricas — como CPU e energia, planejados para o TCC 2 — não requeira modificações nas camadas superiores.

A validação qualitativa nos quatro cenários experimentais confirmou a operacionalidade do sistema: detecção de *jank* durante *rebuilds* intensivos, registro correto de requisições HTTP sob carga de rede, estabilidade do monitoramento de memória com suavização exponencial, e processamento independente de múltiplos tipos de métricas sob carga combinada. As heurísticas de recomendação produziram sugestões contextualmente adequadas em todos os cenários, com latência de detecção inferior ao intervalo de coleta de dois segundos.

Do ponto de vista acadêmico, o trabalho contribui para a linha de pesquisa em instrumentação não-intrusiva de aplicações móveis, propondo uma implementação concreta em Dart com overhead controlado e arquitetura documentada. Do ponto de vista prático, disponibiliza um pacote de código aberto que reduz a barreira de acesso ao diagnóstico de performance para desenvolvedores Flutter sem infraestrutura ou expertise dedicada a profiling.

5.2 Contribuições do TCC 1

As cinco principais contribuições desta fase do trabalho são:

1. **Revisão crítica da literatura:** síntese de estudos sobre monitoramento de recursos em plataformas móveis, identificando a lacuna entre a disponibilidade de ferramentas de diagnóstico e sua adoção contínua durante o desenvolvimento, e fundamentando as escolhas de projeto do protótipo.

2. **Arquitetura modular documentada:** definição e implementação de uma arquitetura de três camadas para sistemas de monitoramento embutido em Flutter, com especificação formal de requisitos funcionais (RF1–RF5) e não funcionais (RNF1–RNF5) e mapeamento explícito entre requisitos, módulos e critérios de avaliação experimental.

3. **Protótipo funcional de código aberto:** implementação dos módulos de coleta, análise e visualização disponibilizados no repositório `BeatrizVocurcaFrade/collector_flutter` incluindo correção de cinco defeitos identificados durante o desenvolvimento: eixo X do gráfico de frames, sincronização do intervalo de memória, escalamento dinâmico do eixo Y, exibição de URLs de rede e tratamento de pico transitórios de memória.

4. **Validação qualitativa em quatro cenários controlados:** execução manual dos cenários de *rebuilds* intensivos, carga de rede, alocação de memória e carga combinada, confirmando operacionalidade do sistema e pertinência das heurísticas implementadas.

5. **Discussão de implicações sociais e ambientais:** análise das dimensões de sustentabilidade digital, inclusão e responsabilidade ética associadas ao monitoramento de desempenho, situando a ferramenta no contexto mais amplo da engenharia de software sustentável.

5.3 Propostas de Continuidade

O TCC 2, previsto para o período de março a julho de 2026, dará continuidade ao trabalho nas seguintes frentes:

- **Implementação de métricas de CPU e energia:** adição de coleta de uso de CPU por processo e estimativa de consumo energético via *platform channels* específicos para Android (`BatteryManager`, `/proc/stat`) e iOS (`UIDevice.batteryLevel`), cobrindo as três métricas ausentes identificadas no TCC 1.

- **Execução formal dos experimentos:** realização dos quatro cenários experimentais em condições controladas, com no mínimo 30 repetições por cenário em três dispositivos físicos (Samsung Galaxy A52, Motorola G73 e emulador Pixel 6). O tamanho da amostra foi calculado com nível de confiança de 95% e poder estatístico de 80%.

- **Análise estatística e comparação com ferramentas de referência:** aplicação de testes t pareados (ou Mann-Whitney em caso de não-normalidade) com intervalos de confiança de 95%; cálculo do erro percentual $|M_{ref} - M_{proto}|/M_{ref} \times 100$ em relação ao Android Profiler e ao `adb shell dumpsys gfxinfo`.

- **Exportação persistente de dados:** implementação da funcionalidade de exportação de métricas em JSON estruturado para armazenamento local, permitindo análise externa e reprodutibilidade das medições.
- **Testes automatizados:** desenvolvimento de suíte de testes unitários para `Analyzer` e `Recommender`, garantindo cobertura das regras heurísticas e prevenindo regressões durante a evolução do código.
- **Avaliação de usabilidade:** coleta de feedback de desenvolvedores Flutter sobre clareza das recomendações, facilidade de integração e utilidade percebida do painel, por meio de protocolo de avaliação baseado na norma ISO 9241-11.
- **Redação final e defesa:** consolidação de todos os capítulos com base nos resultados experimentais e preparação para a defesa pública da monografia.

Referências Bibliográficas

[1] PATHAK, A.; HU, Y. C.; ZHANG, M. *Fine-grained Power Modeling for Smartphones Using System Call Tracing*. In: *Proceedings of the European Conference on Computer Systems*. ACM, 2012.

[2] LIU, Y.; ZHOU, X.; ZHANG, H. *Profiling and Diagnosing Performance Issues in Mobile Applications*. *IEEE Software*, v. 36, n. 5, p. 80–87, 2019.

[3] HE, J.; LI, S.; WANG, Y. *Performance Comparison of Cross-Platform Mobile Development Frameworks*. *Journal of Systems and Software*, v. 152, p. 1–16, 2019.

[4] BANERJEE, S.; ROY, S.; DUTTA, K. *Automated Runtime Monitoring for Mobile Applications*. *ACM Transactions on Software Engineering and Methodology*, v. 29, n. 4, 2020.

[5] HAO, S.; LI, D.; HALFOND, W. *Estimating Mobile Application Energy Consumption Using Program Analysis*. In: *Proceedings of the International Conference on Software Engineering (ICSE)*. IEEE, 2013.

[6] LI, D.; HALFOND, W. *An Empirical Study of Energy Consumption of Android Applications*. In: *International Conference on Software Maintenance and Evolution*. IEEE, 2014.

[7] SILVA, R. A. *Análise de Consumo Energético em Aplicações Móveis Android*. Dissertação (Mestrado em Ciência da Computação) – Universidade Federal de Pernambuco, 2022.

[8] SANTOS, L. F. *Avaliação de Desempenho em Aplicações Flutter: Um Estudo Experimental*. Trabalho de Conclusão de Curso – Universidade de São Paulo, 2023.

[9] RODRIGUES, M. P. *Ferramentas de Monitoramento de Desempenho em Aplicações Móveis*. Trabalho de Conclusão de Curso – Universidade Federal de Minas Gerais, 2021.

[10] ANDROID DEVELOPERS. **Android Profiler**. Disponível em: https://developer.android.com/studio/profile. Acesso em: 05 out. 2025.

[11] ANDROID OPEN SOURCE PROJECT. **Perfetto Tracing**. Disponível em: https://perfetto.dev. Acesso em: 05 out. 2025.

[12] APPLE. **Xcode Instruments**. Disponível em: https://developer.apple.com/xcode/instruments/. Acesso em: 05 out. 2025.

[13] META. **Flipper – React Native Debugging Tool**. Disponível em: https://fbflipper.com. Acesso em: 05 out. 2025.

[14] GOOGLE. **Flutter DevTools**. Disponível em: https://docs.flutter.dev/development/tools/devtools/overview. Acesso em: 05 out. 2025.

[15] GOOGLE. **Flutter: Build Apps for Any Screen**. Disponível em: https://flutter.dev. Acesso em: 22 ago. 2025.

[16] MEYER, J.; KRAUSE, A. *Performance Evaluation of Flutter vs React Native in Production Environments*. *International Journal of Mobile Computing and Multimedia Communications*, v. 8, n. 2, p. 112–125, 2022.

[17] DART TEAM. **Dart: A Client-Optimized Language for Fast Apps on Any Platform**. Disponível em: https://dart.dev. Acesso em: 22 ago. 2025.

[18] CHENG, L.; WANG, X. *Energy Consumption Analysis in Mobile Applications: A Framework-based Approach*. *Journal of Software Engineering*, 15(4), 45–59, 2020.

[19] ZHANG, Y.; LIU, J.; CHEN, F. *Evaluating Energy Efficiency of Flutter and React Native Applications*. In: *Proceedings of the International Conference on Mobile Systems*. ACM, 2021.

[20] JOHNSON, R. *Monitoring System Calls and Frame Rendering Events for Energy Optimization*. *Software: Practice and Experience*, 50(9), 1652–1668, 2020.

[21] KUMAR, P.; GUPTA, R. *Optimizing Resource Consumption in Mobile Apps: Techniques and Tools*. *International Journal of Software Engineering*, 12(4), 234–250, 2019.

[22] GOOGLE SUSTAINABILITY. **Reducing Energy Consumption in Software**. Disponível em: https://sustainability.google. Acesso em: 05 out. 2025.

[23] IEEE. **Sustainability in Software Engineering**. Disponível em: https://www.ieee.org/sustainability. Acesso em: 05 out. 2025.

[24] UNITED NATIONS ENVIRONMENT PROGRAMME (UNEP). **Digital Sustainability Initiatives**. Disponível em: https://www.unep.org/resources. Acesso em: 05 out. 2025.

[25] GIL, A. C. **Métodos e Técnicas de Pesquisa Social**. 7. ed. São Paulo: Atlas, 2017.

[26] LAKATOS, E. M.; MARCONI, M. A. **Fundamentos de Metodologia Científica**. 9. ed. São Paulo: Atlas, 2021.

[27] YIN, R. K. **Estudo de Caso: Planejamento e Métodos**. 5. ed. Porto Alegre: Bookman, 2015.

[28] WOHLIN, C. et al. **Experimentation in Software Engineering**. Springer, 2012.

[29] PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software: Uma Abordagem Profissional**. 9. ed. Porto Alegre: AMGH, 2020.

[30] KITCHENHAM, B.; CHARTERS, S. **Guidelines for Performing Systematic Literature Reviews in Software Engineering**. EBSE Technical Report, Keele University, 2007.

[31] SOMMERVILLE, I. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2019.

[32] MARTIN, R. C. **Clean Architecture: A Craftsman's Guide to Software Structure and Design**. Prentice Hall, 2018.

[33] EVANS, E. **Domain-Driven Design: Tackling Complexity in the Heart of Software**. Addison-Wesley, 2004.

[34] RAVINDRANATH, L.; PADHYE, J.; AGARWAL, S.; MAHAJAN, R.; OBERMILLER, I.; SHAYANDEH, S. *AppInsight: Mobile App Performance Monitoring in the Wild*. In: *Proceedings of the 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*. USENIX Association, 2012. p. 107–120.

[35] VERMA, A.; TRIVEDI, A.; PANDEY, H. M. *A large-scale empirical study on mobile performance: energy, run-time and memory*. *Empirical Software Engineering*, v. 28, art. 161, 2023. DOI: 10.1007/s10664-023-10391-y.

[36] HORN, R. et al. *Native vs Web Apps: Comparing the Energy Consumption and Performance of Android Apps and their Web Counterparts*. In: *Proceedings of the 10th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*. ACM, 2023.

[37] DORNAUER, B.; FELDERER, M. *Energy-Saving Strategies for Mobile Web Apps and their Measurement: Results from a Decade of Research*. In: *Proceedings of the 10th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*. ACM, 2023.

[38] SUN, Y. et al. *Energy inefficiency diagnosis for Android applications: a literature review*. *Frontiers of Computer Science*, v. 17, n. 1, art. 171201, 2023. DOI: 10.1007/s11704-021-0532-4.

[39] TONI, C. et al. *Energy-Consumption Estimation of API-usage in Smartphone Apps via Static Analysis*. In: *Proceedings of the 20th International Conference on Mining Software Repositories (MSR)*. IEEE, 2023. DOI: 10.1109/MSR59073.2023.00057.